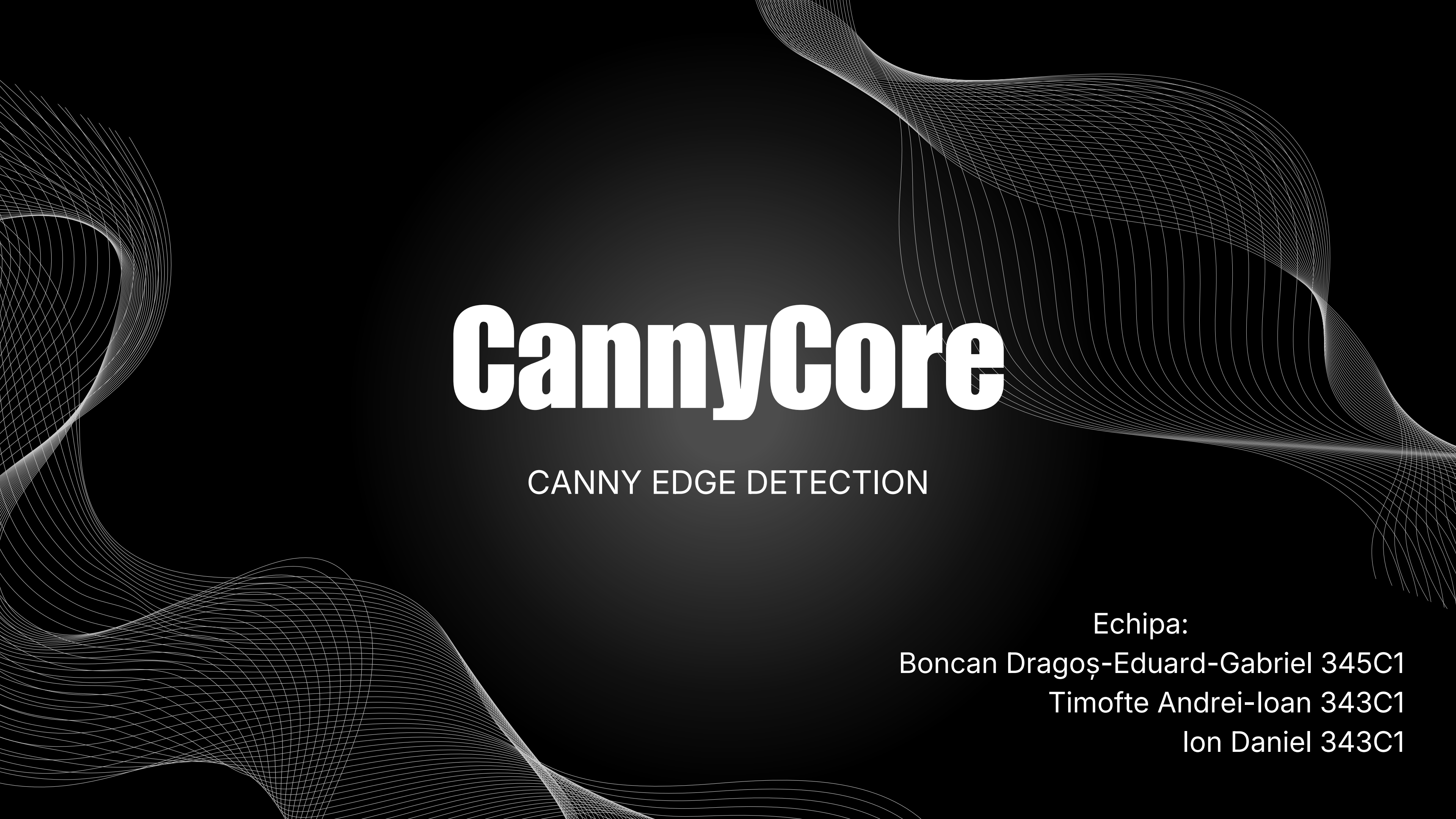




CannyCore

CANNY EDGE DETECTION

Echipa:

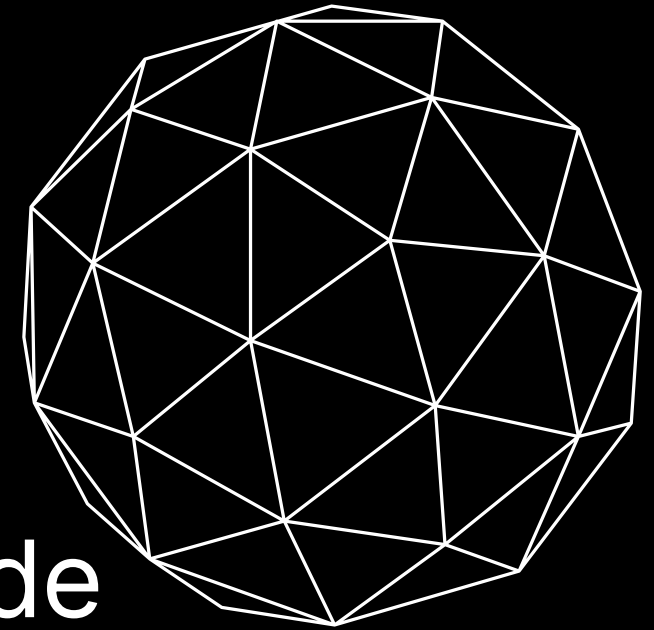
Boncan Dragoș-Eduard-Gabriel 345C1

Timofte Andrei-Ioan 343C1

Ion Daniel 343C1



Cuprins



I. Varianta Serială

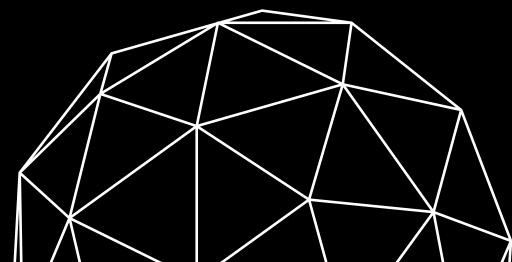
IV. Variante Hibride

II. Variante Multithreaded

V. Analiză Comparativă

III. Varianta Multiprocessing

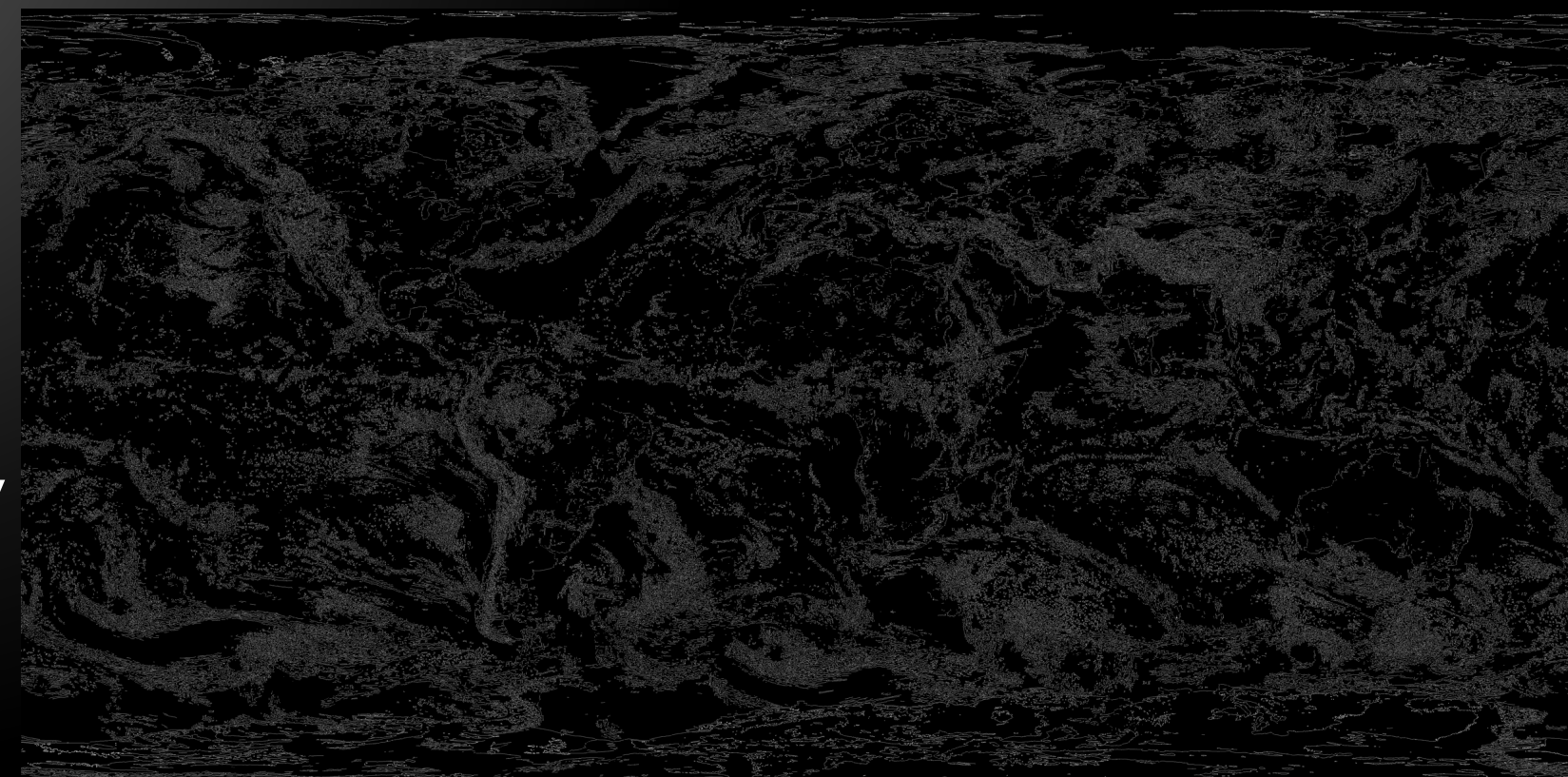
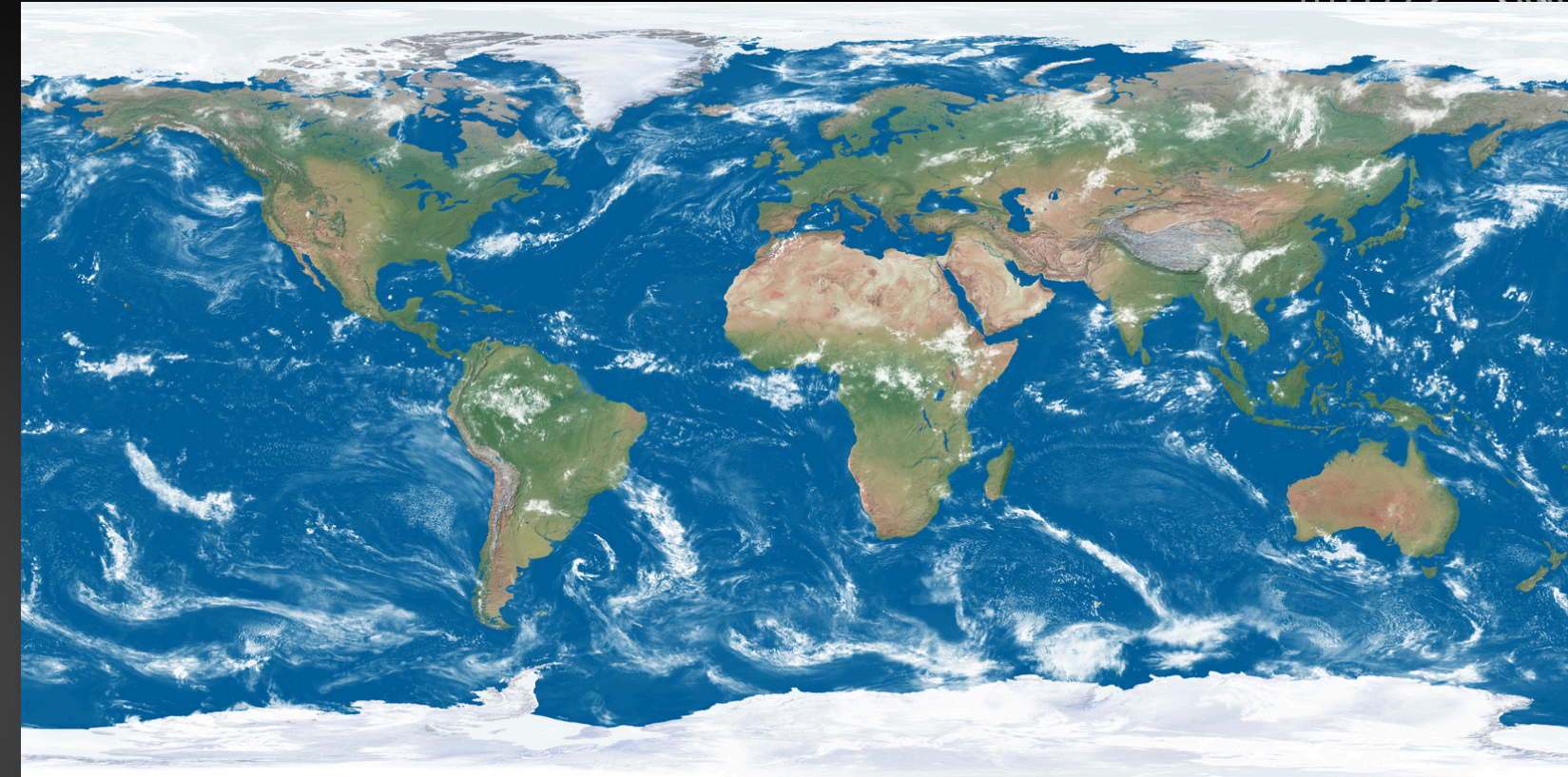
VI. Concluzii



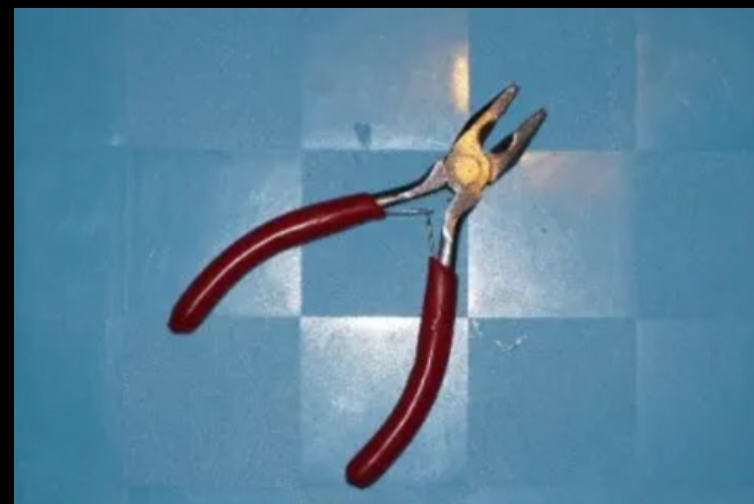
Canny Edge Detection

Etape:

- **Conversia Grayscale:** Transformarea imaginii din RGB într-o imagine cu un singur canal de intensitate
- **Filtrului Gaussian:** Reducerea zgomotului din imagine.
- **Calculul Gradientului(Sobel):** Aplicarea a două măști de convoluție pentru a detecta intensitatea și direcția conturilor.
- **Non-Maximum Suppression:** Reducerea muchiilor la o grosime de un singur pixel.
- **Hysteresis Edge Tracking:** Finalizarea detecției conturilor, păstrându-se doar pixelii weak dacă sunt conectați la un pixel strong



Canny Edge Detection



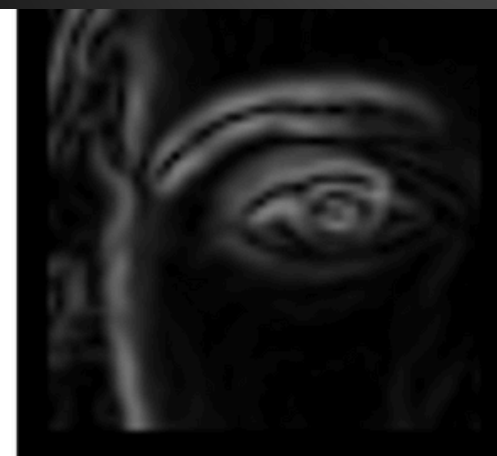
SOBEL



NON-MAXIMUM
SUPPRESSION



(a) Gaussian
smoothing



(b) Sobel edge
detection



(c) non-maximum
suppression



(d) hysteresis
thresholding

Stages in Canny Edge Detection

Profiling algoritm serial

Elapsed Time [?]: 9.984s >

CPU Time [?]: 9.210s
Total Thread Count: 3
Paused Time [?]: 0s

Top Hotspots >

This section lists the most active functions in your application. Optimizing these hotspot functions typically results in improving overall application performance.

Function	Module	CPU Time [?]	% of CPU Time [?]
Image::gaussianBlur	Canny_contur	4.480s	48.6%
sobelGradient	Canny_contur	0.920s	10.0%
atan2f	libm.so.6	0.850s	9.2%
stbi_zlib_compress	Canny_contur	0.540s	5.9%
nonMaxSuppression	Canny_contur	0.480s	5.2%
[Others]	N/A*	1.941s	21.1%

*N/A is applied to non-summable metrics.

Identificarea Hotspot-urilor:

- **Tool folosit:** Intel VTune Profiler
- **Baseline de Performanță:** Timp total 9.98s. Aceasta este valoarea de referință (T_serial) folosită pentru a calcula accelerarea (Speedup) celorlalte variante.
- **Bottleneck Principal:** Funcția Image::gaussianBlur este responsabilă pentru 48.6% din timp. În varianta serială, aceasta este pur "CPU Bound" (limitată strict de frecvența procesorului).
- **Structura Costurilor:** Următoarele funcții consumatoare sunt sobelGradient (10.0%) și funcția matematică atan2f (9.2%).
- **Lipsa Overhead-ului:** În cazul algoritmului serial nu apar funcții de sistem (comunicare rețea, creare thread-uri), deci tot timpul procesorului este dedicat logicii algoritmului.

Varianta Pthreads

Inovații & Diferențe:

- **Data Parallelism:** Procesare pe benzi orizontale per thread.
- **Structuri dedicate:** Argumente împachetate pentru funcțiile void*.
- **Sincronizare:** Bariere pthread_join între etapele majore.
- **Hysteresis Hibrid:** Thresholding paralel, dar tracking (DFS) secvențial.

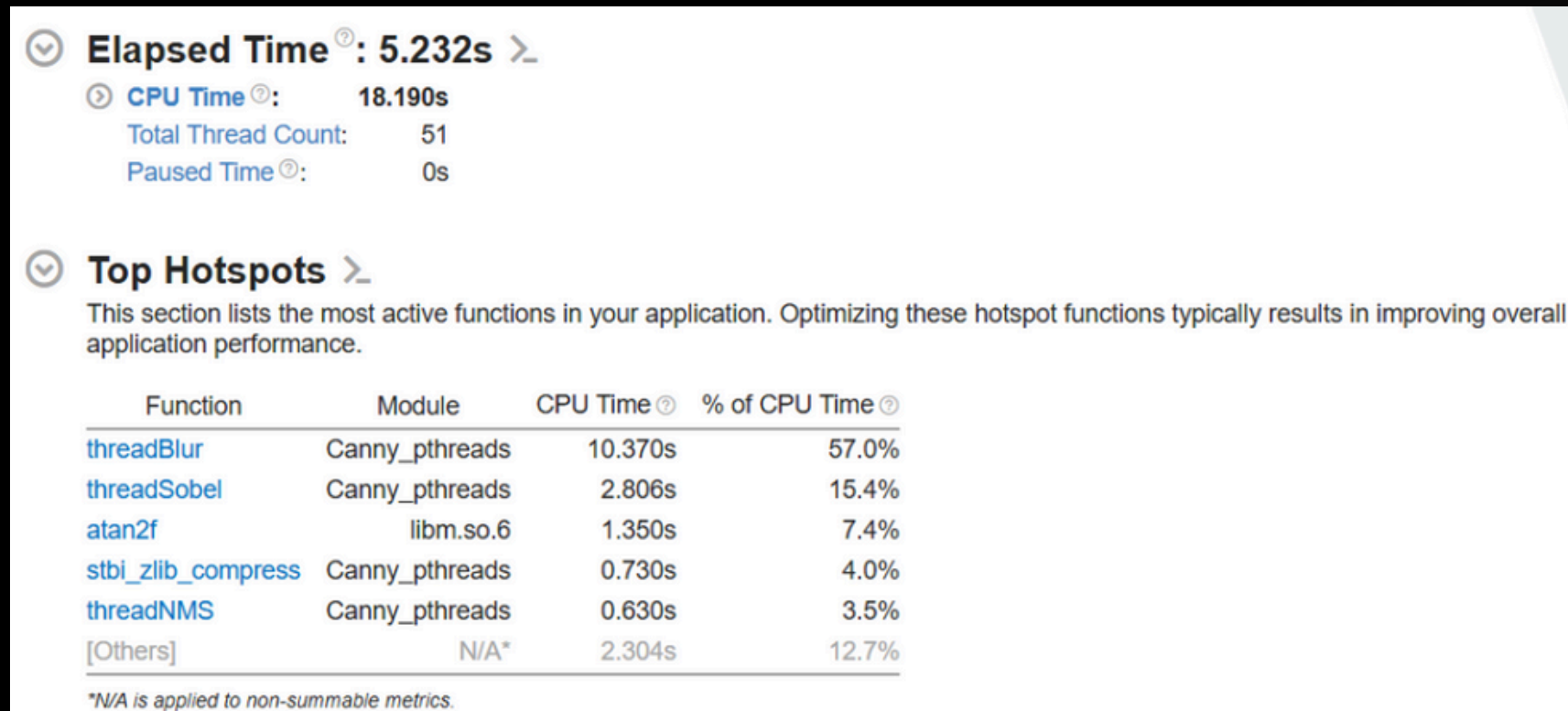
Probleme întâmpinate:

- **Limitare Amdahl:** Hysteresis rămâne secvențial (DFS recursiv).
- **Gestionare Indici:** Calcul manual complex pentru limite și margini.
- **Overhead Thread-uri:** Lansare și unire repetată per etapă.
- **Lipsă SIMD:** Procesare scalară, fără vectorizare (AVX/SSE).

Flux de date:

Memorie Partajată (RAM) -> Procesare Paralelă (CPU) -> Sincronizare

Profiling Pthreads

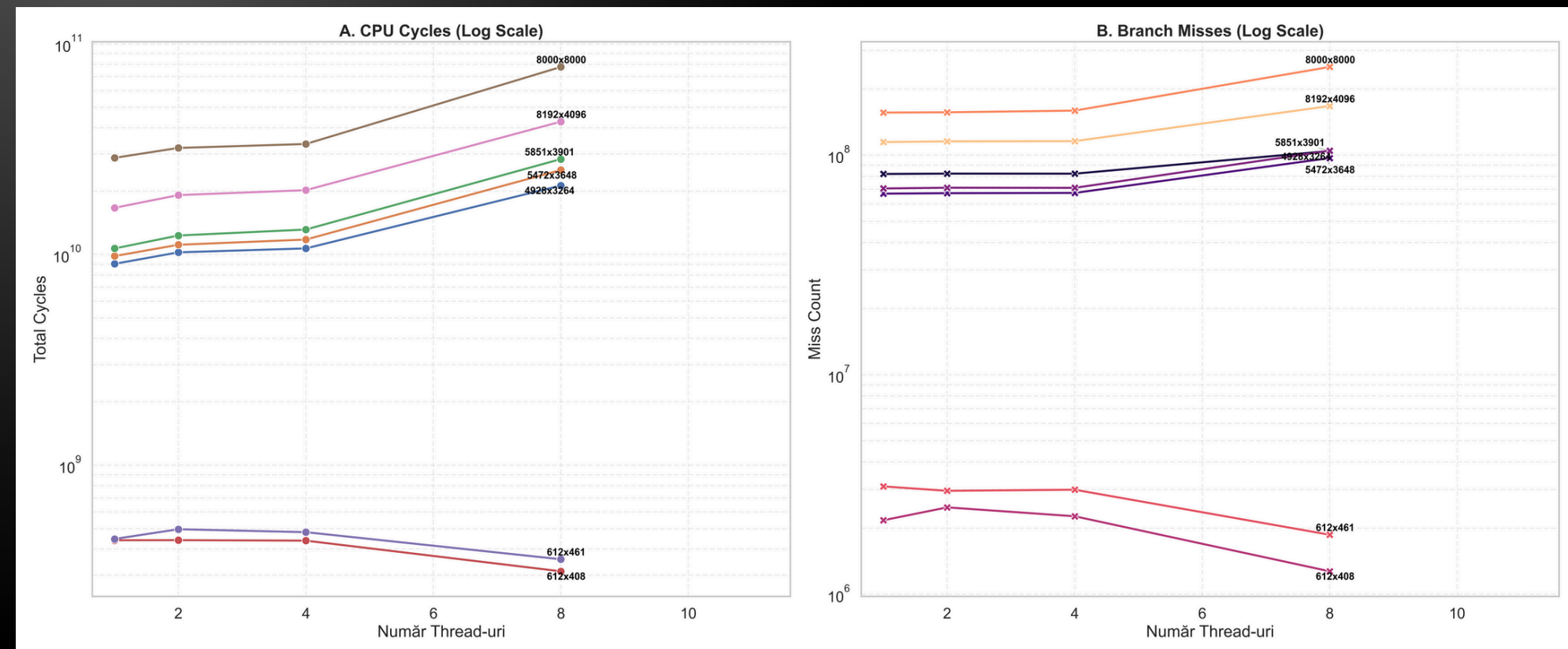


Performanțe hardware:

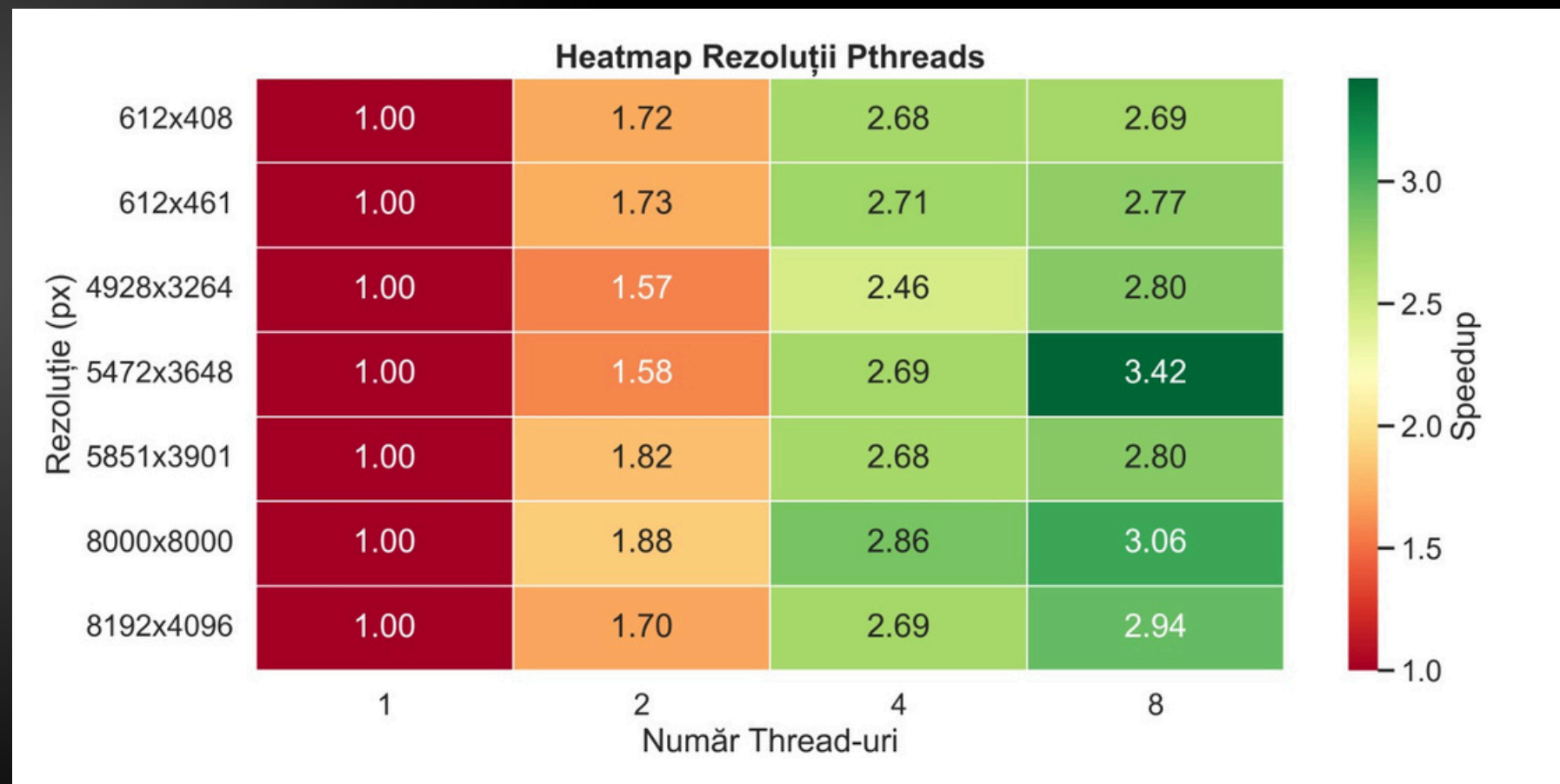
- **Tool folosit:** perf rulat pe xl
- **Costul Paralelizării:** La imagini mari, efortul de coordonare a thread-urilor se cumulează, crescând numărul total de cicli executați.
- **Anomalia Frecvenței:** La imagini mici, ciclurile scad artificial din cauza reducerii frecvenței CPU și a stărilor de repaus (sleep).
- **Predictibilitate:** Sarcina infimă per thread la imagini mici simplifică drastic predicția ramurilor, eliminând aproape total erorile (misses).

Identificarea Hotspot-urilor:

- **Tool folosit:** Intel VTune Profiler
- **Bottleneck principal:** Funcția threadBlur este responsabilă pentru 57% din timpul total de procesare (10.37 secunde pe CPU).
- **Procesare Gradient:** Etapa threadSobel consumă 15.4% din resurse.
- **Cost Matematic:** Funcția de bibliotecă atan2f (calculul direcției gradientului) ocupă un surprinzător 7.4%, fiind a treia cea mai costisitoare operație.
- **I/O și Compresie:** Scrierea imaginii are un impact minor (4.0%).



Rezultate Pthreads

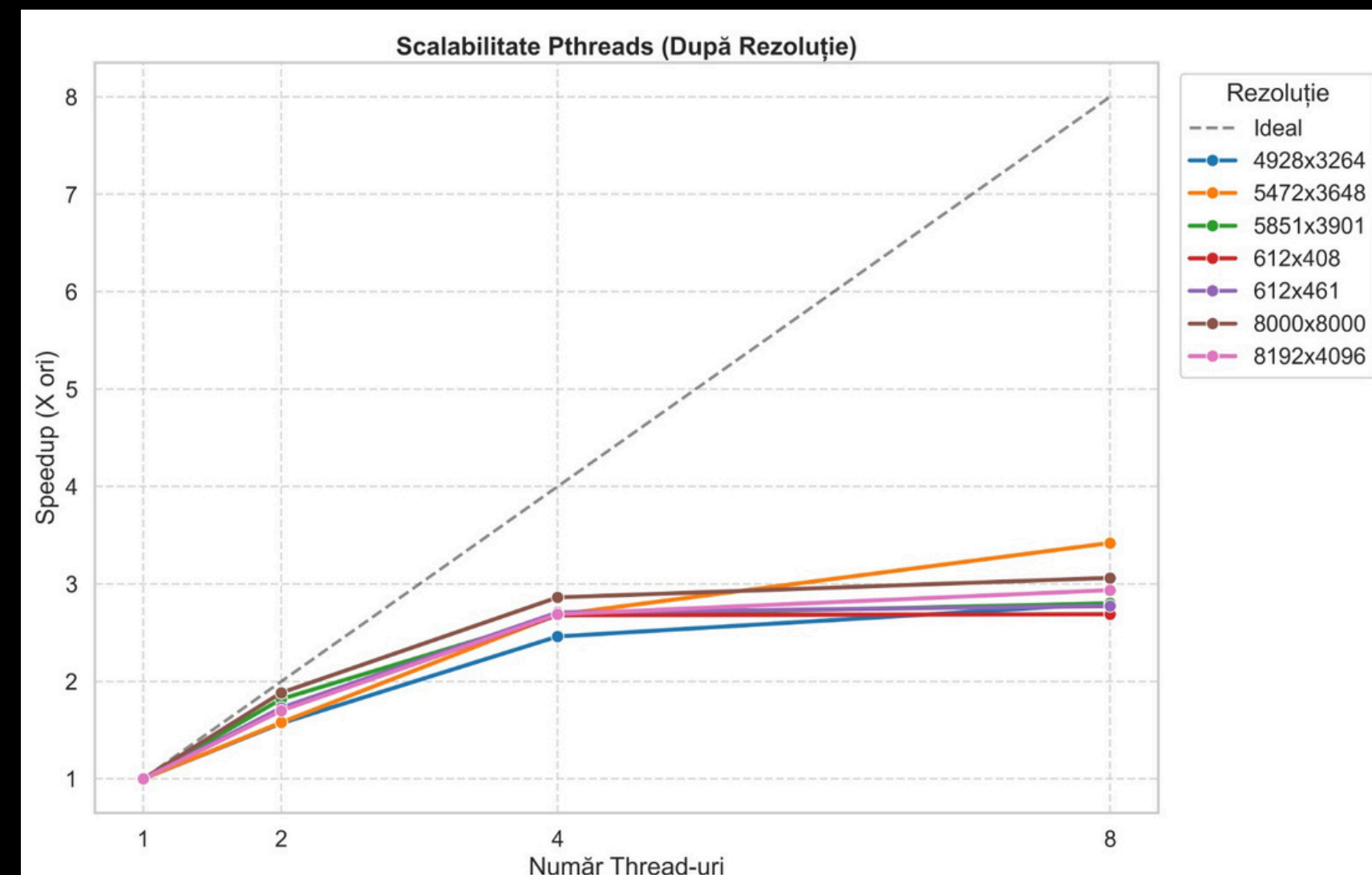


Rezultatul Optim:

- Speedup: 3.42x
- Configurație: 8 Thread-uri pentru poză de rezoluție 5472x3648 pixeli.
- Concluzie: Eficiența maximă nu este la rezoluția cea mai mare, ci acolo unde datele încap cel mai bine în cache-ul procesorului, minimizând accesul la RAM.
- Eficiență maximă de 94% pentru 2 threaduri pentru 1_earth_8k (8192x4096)

Scalabilitate Sub-liniară:

- Observăm o accelerare bună până la 4 thread-uri (~2.7x - 2.8x).
- La 8 thread-uri, curba se plafonează (saturație), atingând un maxim de 3.42
- Cu cât este mai mare poza, cu atât speedup-ul este mai bun. Acest lucru este datorat păstrării aceluiași overhead pentru mai mult lucru.



Varianta OpenMP

Inovații & Diferențe:

- **Abstractizare Înaltă:** Utilizarea directivelor `#pragma omp parallel for` elimină complet codul "boilerplate" (creare/join manual de thread-uri).
- **Automatic Scheduling:** Distribuția automată a iterațiilor (loop chunks) pe nuclee, eliminând calculul manual de indici (hStart, hEnd).
- **Structură Simplificată:** Nu mai sunt necesare structuri auxiliare (struct Args) pentru împachetarea argumentelor funcțiilor.
- **Sincronizare Implicită:** Bariere automate la finalul fiecărei regiuni paralele (nu necesită apeluri explicite de join între etape).

Probleme întâmpinate:

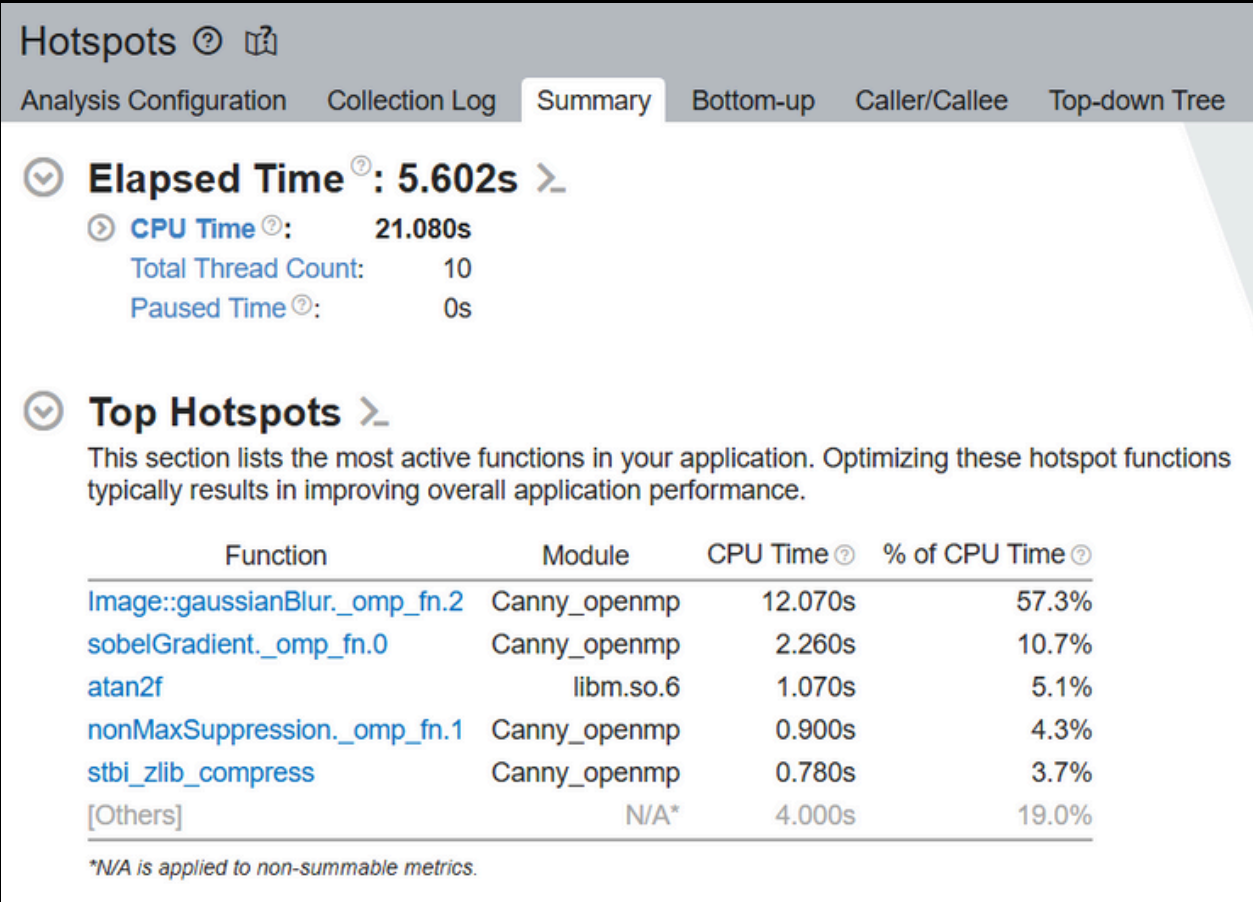
- **Limitare Amdahl:** Tracking-ul muchiilor (DFS recursiv) rămâne un bottleneck secvențial (difícil de paralelizat eficient doar cu `#pragma`).
- **Overhead Fork-Join:** Costul repetat de creare și distrugere a echipelor de thread-uri pentru fiecare funcție din pipeline (Sobel, NMS, Threshold).
- **Granularitate:** Dacă "chunks" sunt prea mici (scheduling dinamic excesiv), overhead-ul de management depășește câștigul de viteză.
- **Control Fin:** Mai puțin control asupra afinității thread-urilor comparativ cu Pthreads (risc de cache thrashing în scenarii specifice).

Flux de date:

Master Thread -> Fork -> Procesare Paralelă -> Join (Implicit Barrier)

Profiling OpenMP

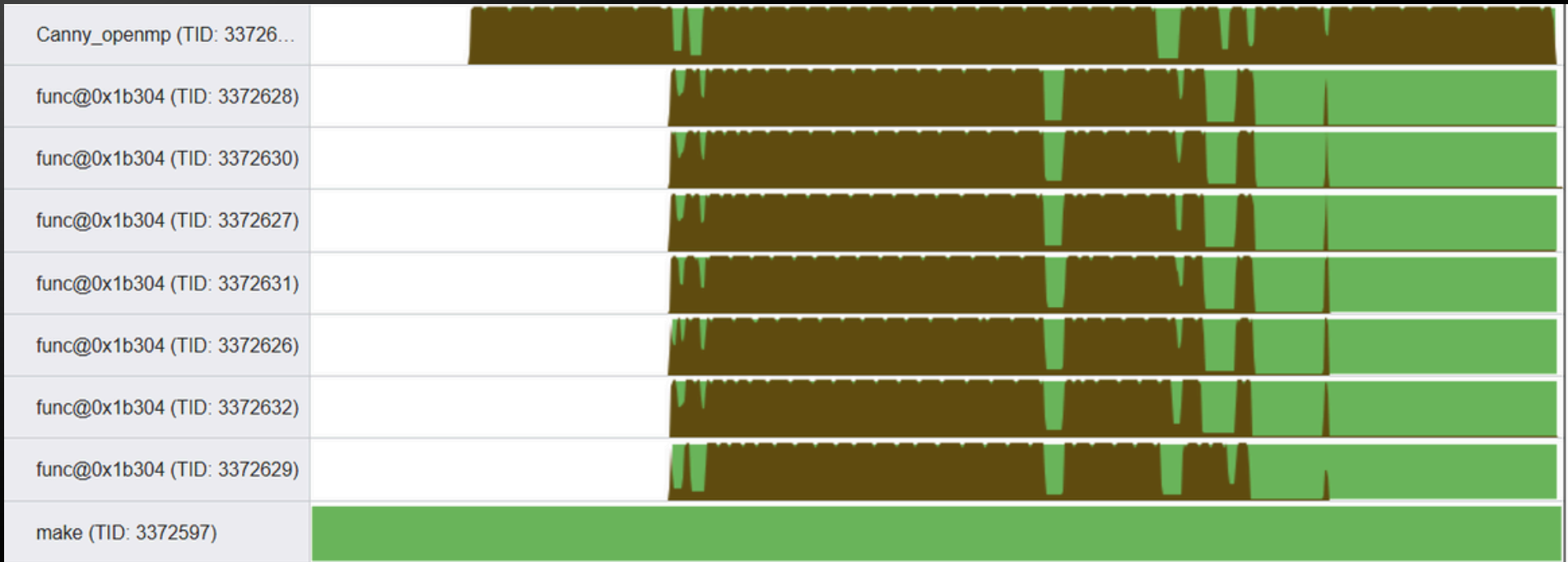
Analiză Cantitativă (Hotspots & Timpi)



- **Tool folosit:** Intel VTune Profiler
- **Identificarea Bottleneck-ului:** Funcția gaussianBlur este consumatorul principal (57.3% din CPU Time), fiind candidatul ideal pentru optimizare paralelă pe imaginea 8K (~33MP).
- **Eficiența Paralelizării:** Diferența dintre CPU Time (21.08s) și Elapsed Time (5.60s) confirmă că efortul total de calcul a fost împărțit simultan între nuclee, reducând durata reală de așteptare de aproape 4 ori.
- **Impactul Secundar:** Funcții precum sobelGradient contribuie cu 10.7%, restul timpului fiind distribuit pe funcții auxiliare sau secvențiale, inclusiv citirea și salvarea pozei.

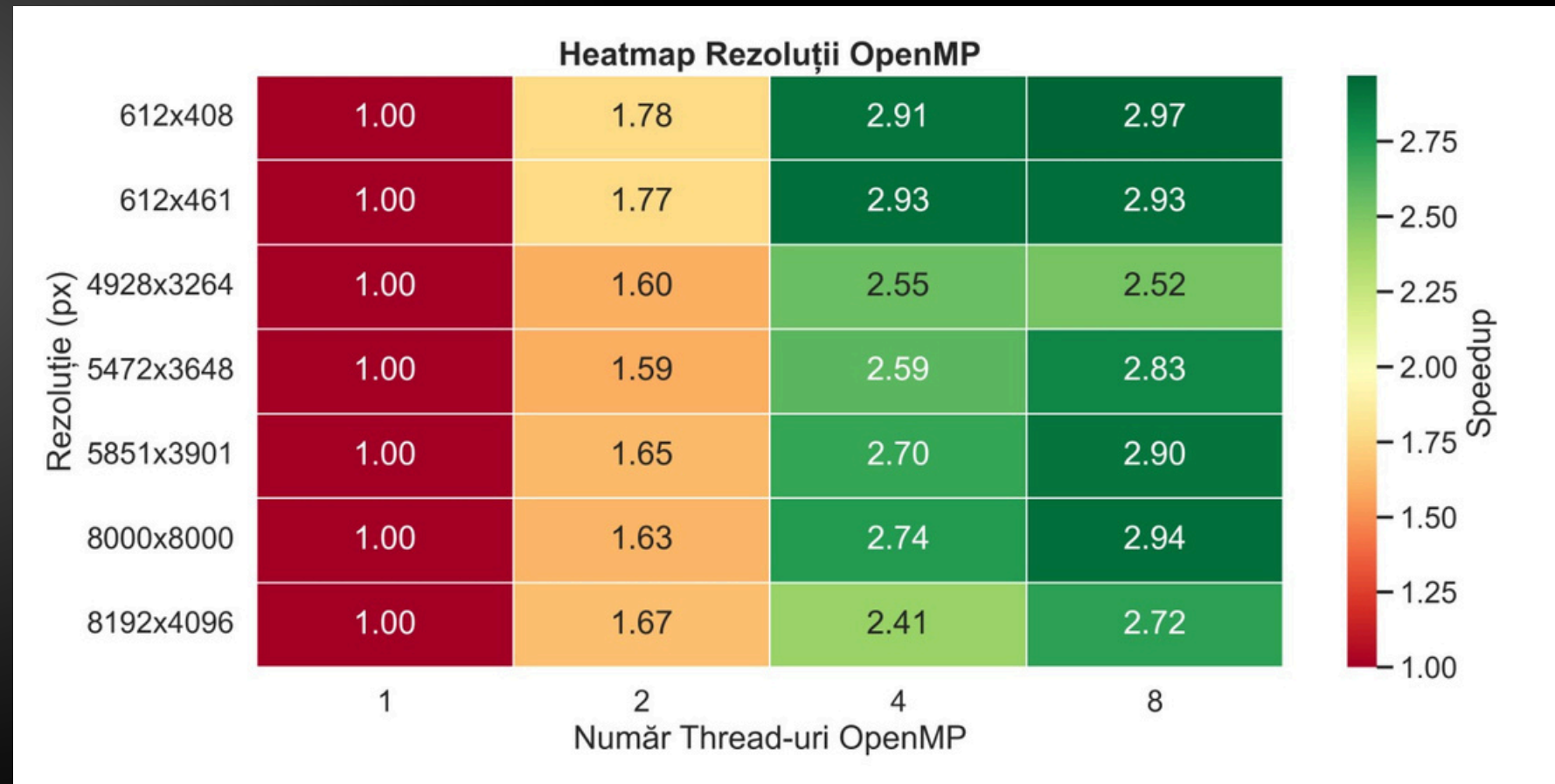
Analiză Vizuală (Thread Timeline)

- **Activitate:** Verde indică execuția efectivă, iar Maro reprezintă timpii de așteptare (Overhead) și zonele seriale.
- **Load Balancing:** Lungimile egale ale benzilor verzi confirmă distribuția uniformă a sarcinii pe cele 8 thread-uri.
- **Sincronizare:** Graficul evidențiază clar momentele de aliniere (barierele) la intrarea și ieșirea din paralelism.



Rezultate OpenMP

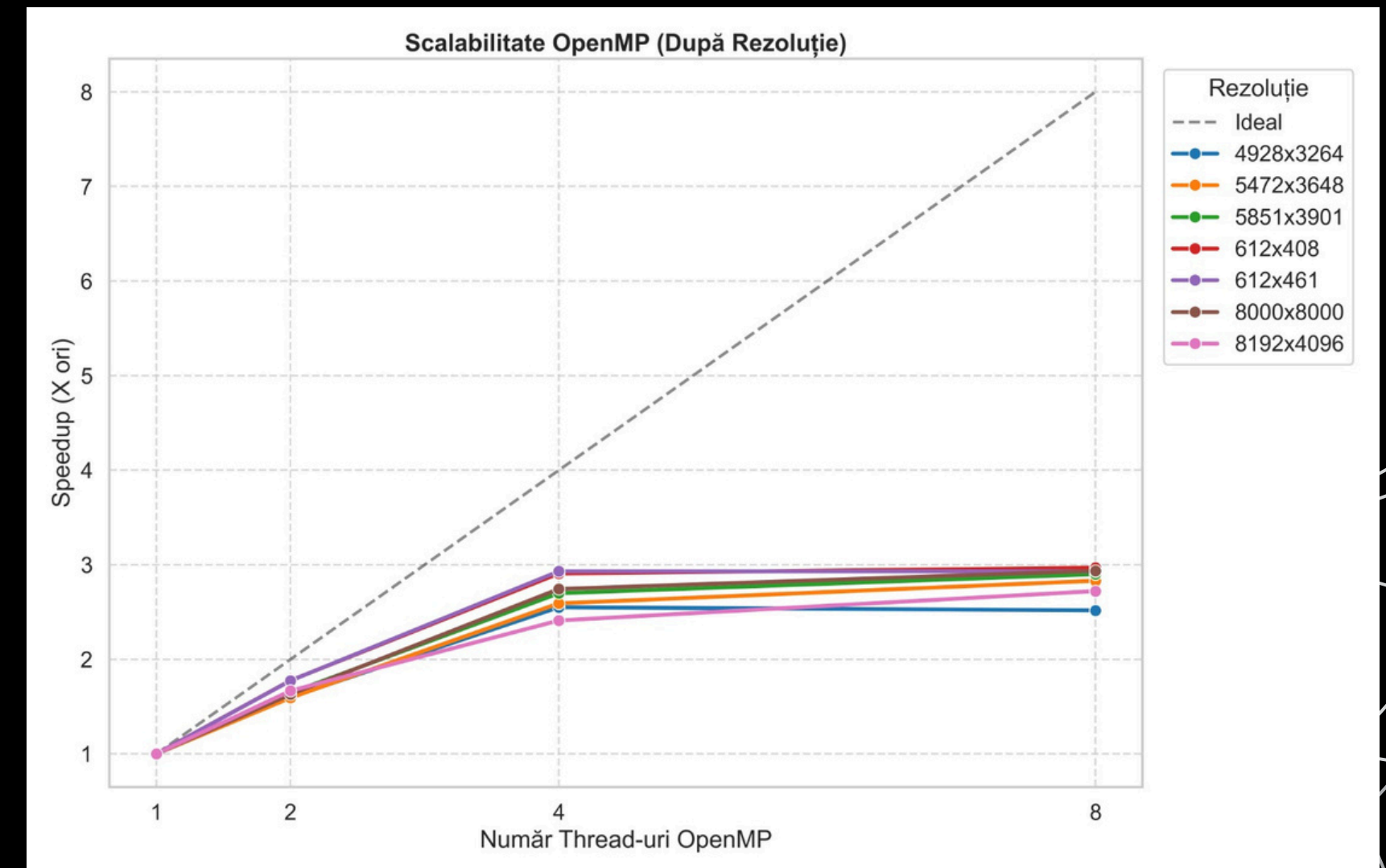
Scalabilitate & Overhead:



- Plafonare la 4 Thread-uri:
- Creșterea de performanță este solidă până la 4 nuclee (~2.9x), dar curba se aplatizează mai devreme decât la Pthreads.
- La 8 thread-uri, câștigul este marginal sau chiar negativ în unele cazuri (ex: 4928px scade de la 2.55 la 2.52).

Rezultat Maxim:

- 2.97x (la 612x408 px)
- Variația performanței între rezoluții este mai mică, sugerând un scheduling intern robust, dar conservator.
- OpenMP favorizează simplitatea codului în detrimentul controlului fin al performanței absolute.
- Eficiență maximă de 89% pentru 2 threaduri.



Varianta CUDA

Inovații & Diferențe:

- **Mapare directă** 1 thread / pixel
- **Kerneluri dedicate** per etapă
- **Shared memory** pentru procesarea kernelului Gaussian
- **Eliminarea recursivității** în etapa de hysteresis
- **Procesare integrală** a pipeline-ului pe GPU

Probleme întâmpinate:

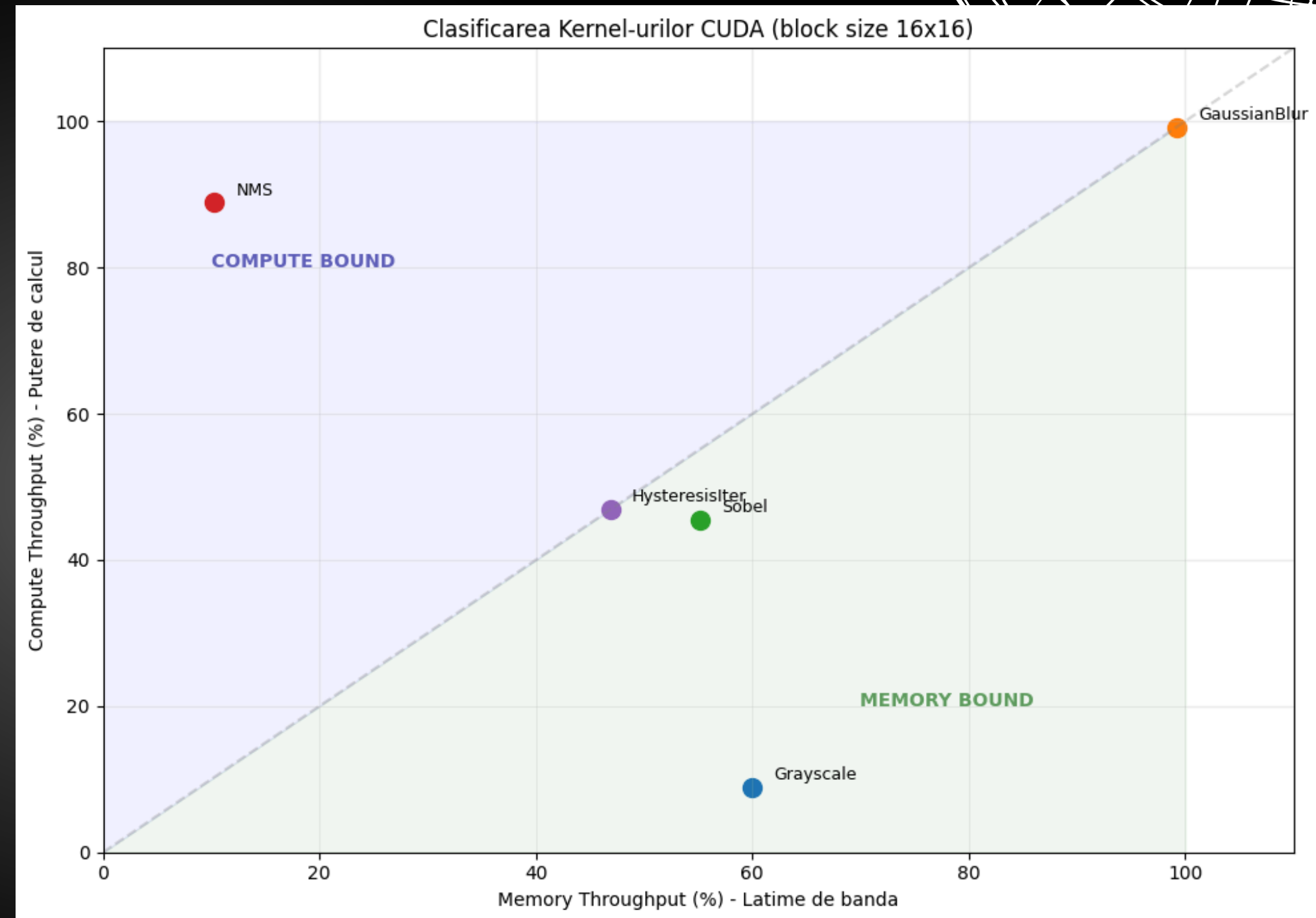
- **Hysteresis pe GPU**, implementarea unui mecanism de promovare a muchiilor weak fără recursivitate
- **Memory management**, alocarea și eliberarea datelor de pe CPU & GPU
- **Limitări shared memory**, în cazul nostru procesarea filtrului Gaussian este destul de lightweight

Flux Date:

CPU (load & transfer) → GPU (procesare date) → CPU (final)

Profiling CUDA

- **Tool folosit:** Nsight Compute pe un RTX 2000 ADA Gen
- **Eficiență maximă** atinsă de filtrul Gaussian, 99.18% Throughput
- Hysteresis & Sobel **sunt limitate de natura lor algoritmică** (multe ramificări logice)
- **NMS** prezintă cele mai complexe calcule per pixel, 88.9% Compute Throughput



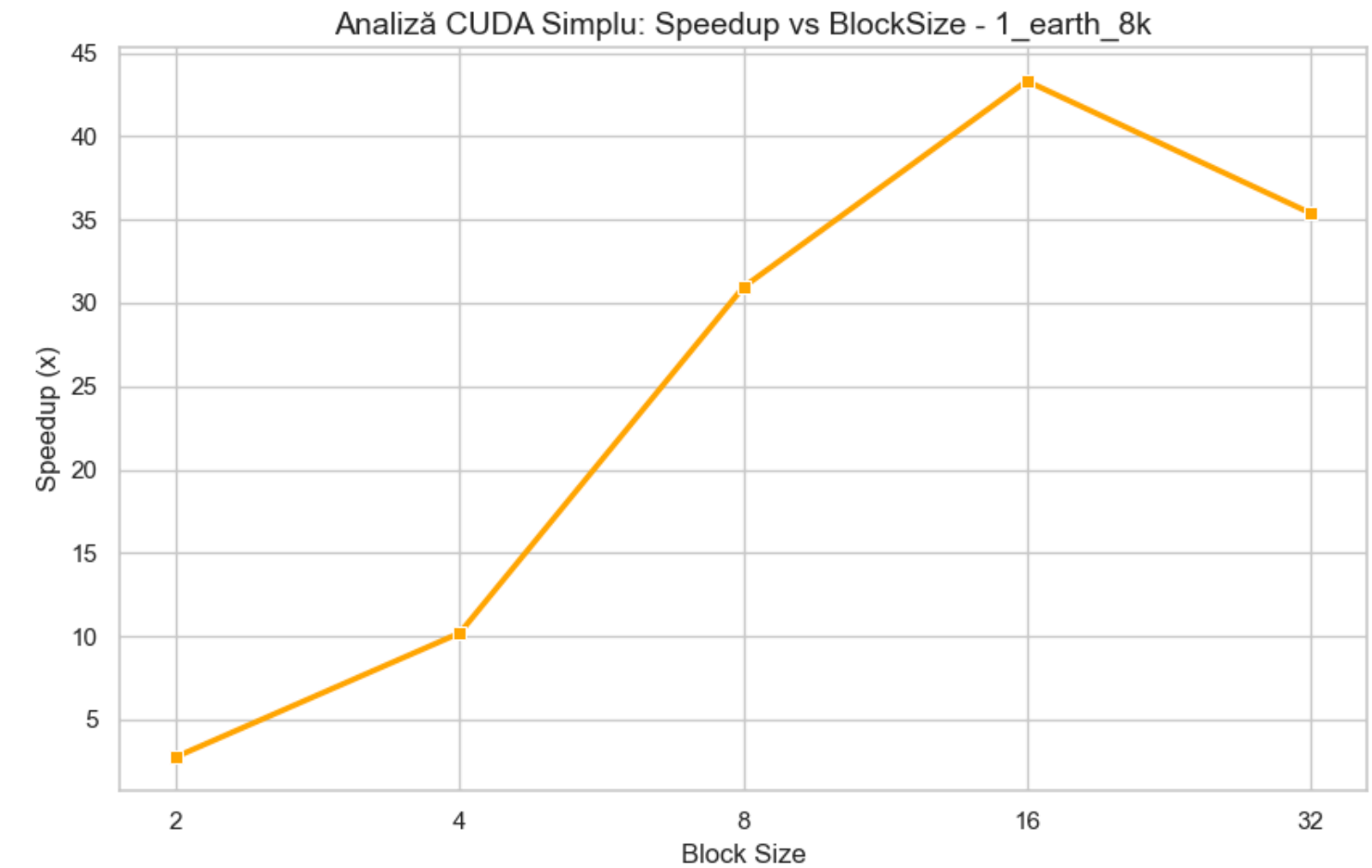
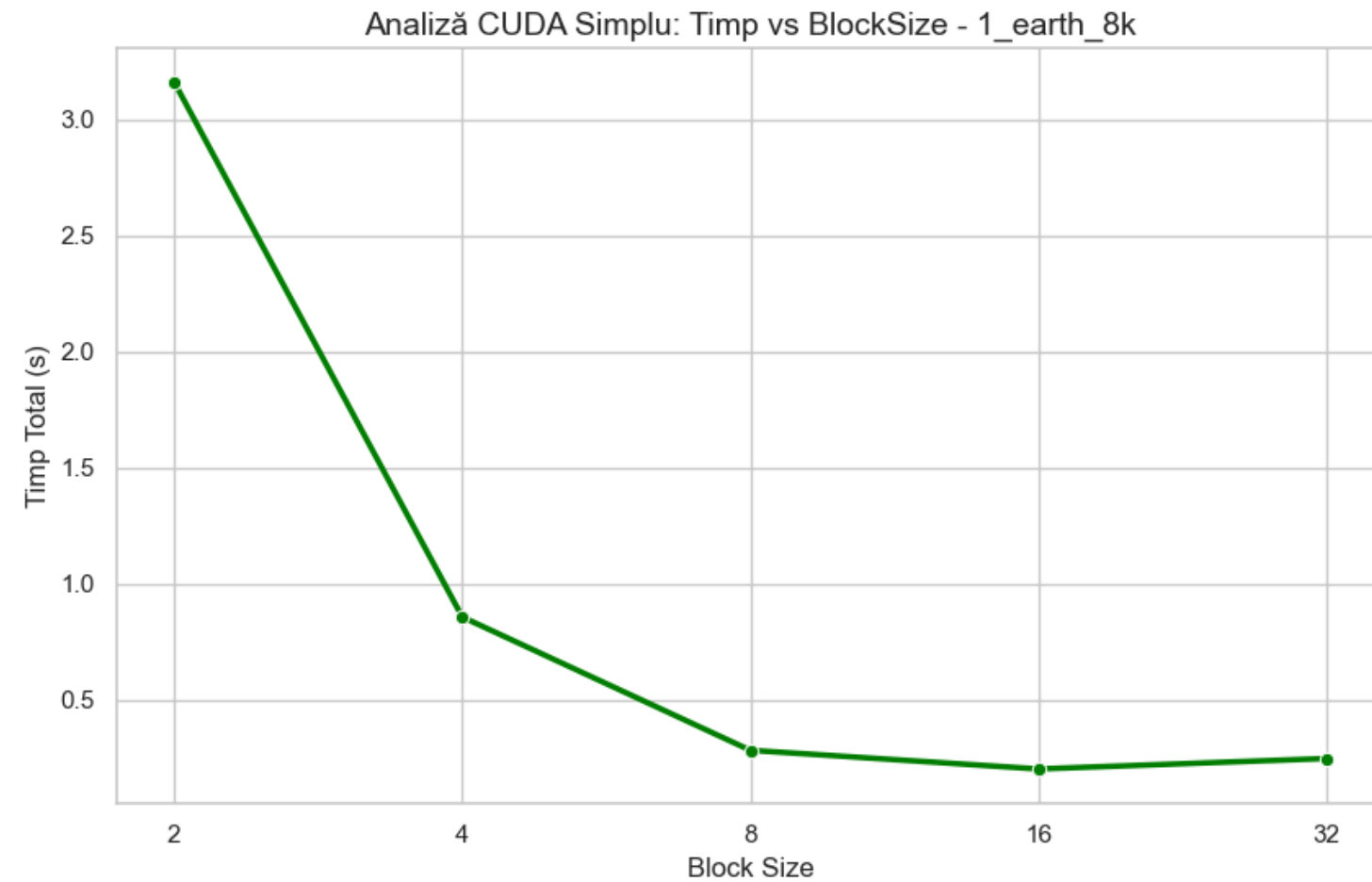
Complexitate Spațială: $O(H \times W)$

Complexitate Temporală: $O(I \times H \times W / P) \sim O(1)$

H = image height, W = image width,

P = num procs CUDA, I = iters hysteresis

Rezultate CUDA



Configurație optimă: 16x16 (256 threads per block)

Cel mai bun timp obținut: 0.178s

Speedup obținut: 44.58x

Eficiență obținută: 79.6% (56 cores)

Varianta MPI

Inovații & Diferențe:

- **Paralelismul datelor:** Împărțirea imaginii în benzi orizontale pentru fiecare proces.
- **Gaussian blur separabil** - Blur-ul este optimizat prin separare, astfel blur-ul 2D este aplicat ca blur 1D orizontal urmat de blur 1D vertical.
- **Hysteresis Hibrid:** Thresholding paralel, dar tracking (DFS) secvențial.

Probleme întâmpinate:

- **Comunicația între procese** - Schimbul de date dintre procese are overhead semnificativ.
- **Dependențele la margini** - Pixelii de la marginea superioară a unei sub-imagini au vecini care se află în sub-imaginea procesului anterior. Calculul corect al gradientilor necesită comunicație suplimentară între procese.

Flux date:

Master -> MPI Scatter -> MPI (CPU) -> MPI Gather -> Master

Profiling MPI

Elapsed Time[Ⓢ]: 12.371s >

CPU Time[Ⓢ]: 58.380s

Total Thread Count: 44

Paused Time[Ⓢ]: 0s

Top Hotspots >

This section lists the most active functions in your application. Optimizing these hotspot functions typically results in improving overall application performance.

Function	Module	CPU Time [Ⓢ]	% of CPU Time [Ⓢ]
MPI_Bcast	libmpi.so.40	21.510s	36.8%
Image::gaussianBlur	Canny_mpi	8.969s	15.4%
sobelGradient	Canny_mpi	3.890s	6.7%
ompi_mpi_finalize	libmpi.so.40	3.487s	6.0%
stbiw_zlib_countm	Canny_mpi	1.360s	2.3%
[Others]	N/A*	19.164s	32.8%

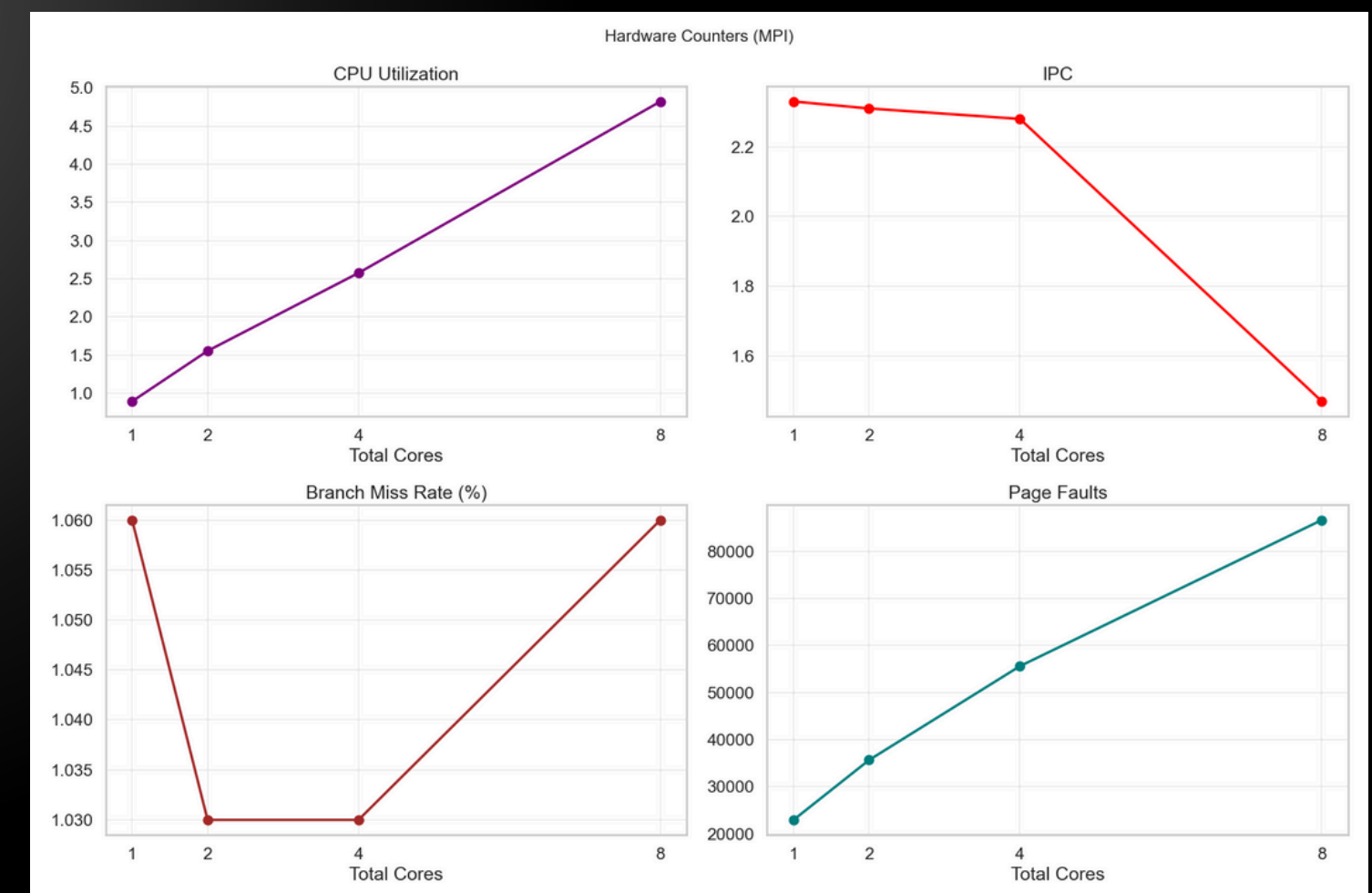
*N/A is applied to non-summable metrics.

Performanțe hardware:

- **Tool folosit:** perf rulat pe xl
- **Utilizare vs. Eficiență (IPC):** Graficul Instructions Per Cycle scade drastic pe măsură ce creștem numărul de core-uri (în special la 8 core-uri), deși CPU Utilization crește liniar. Asta înseamnă că procesorul stă mult în stări de așteptare (busy-waiting pentru date de la MPI), nu făcând calcule utile.
- **Izolarea Memoriei (Page Faults):** Graficul Page Faults arată o creștere liniară perfectă. În MPI fiecare proces își alocă propria memorie și copie a datelor, ducând la un overhead de sistem semnificativ mai mare.
- **Stabilitatea Execuției:** Branch Miss Rate crește brusc la 8 nuclee, sugerând că logica de sincronizare între multe procese devine impredictibilă pentru procesor.

Identificarea Hotspot-urilor:

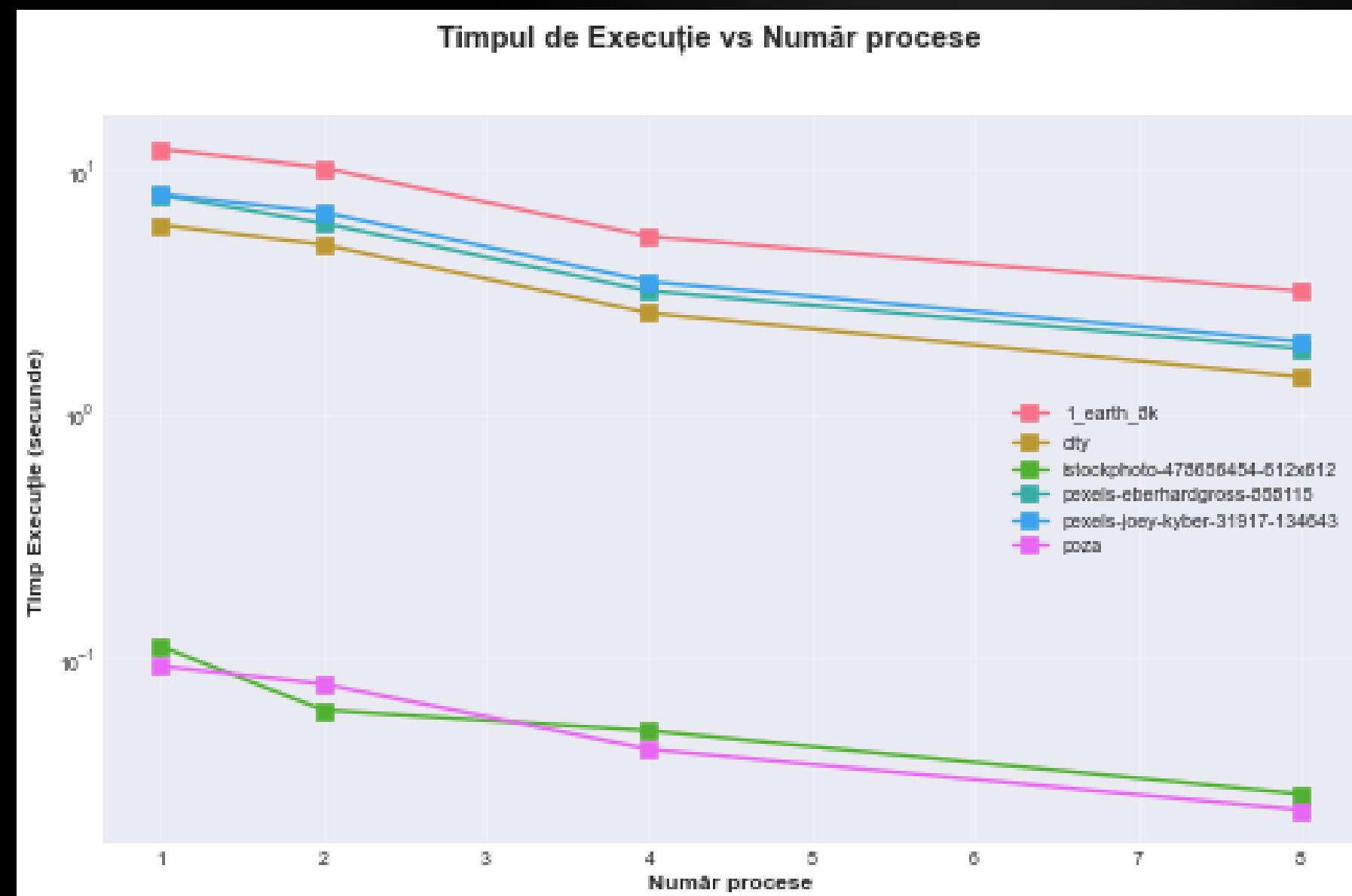
- **Tool folosit:** Intel VTune Profiler
- **Bottleneck principal:** Funcția MPI_Bcast este dominantă, ocupând 36.8% din timpul total (21.5s). Asta indică faptul că distribuirea datelor către procese costă mai mult decât procesarea lor efectivă.
- **Procesare Efectivă:** Funcția Image::gaussianBlur (kernel-ul de calcul) a scăzut pe locul doi, reprezentând 15.4% din timp.
- **Overhead MPI:** Se observă costuri administrative specifice, precum ompi_mpi_finalize (6.0%), reprezentând timpul necesar pentru sincronizarea și închiderea mediului distribuit.



Rezultate MPI

Rezultatul Optim:

- **Configurația Optimă:** 8 procese. Graficul arată cea mai mare scădere a timpului de execuție în acest punct pentru toate dataset-urile mari.
- **Cel mai bun timp:** Aproximativ 5 secunde pentru cel mai complex dataset (1_earth_8k), o îmbunătățire majoră față de cele ~14 secunde inițiale.
- **Speedup:** Se observă o accelerare de aproape 4x la utilizarea a 8 procese comparativ cu execuția pe un singur proces (pentru imaginile mari).
- **Eficiență:** Scalarea este puternică la imaginile mari. La cele mici (liniile din partea de jos), curba se aplatizează rapid, semn că overhead-ul comunicării MPI anulează beneficiile paralelizării.



Varianta MPI-OPENMP

Inovații & Diferențe:

- **Arhitectură pe Două Niveluri:** Distribuția datelor se face prin MPI, iar calculul intensiv este paralelizat local prin OpenMP
- **Scheduling Adaptiv:** Utilizează schedule static pentru operații uniforme și dinamic pentru cele variabile
- **Optimizări Algoritmice:** Kernel-ul Gaussian este descompus pentru reducerea complexității, iar recursivitatea din Hysteresis este înlocuită cu iterații paralele.

Probleme întâmpinate:

- **Overhead** - Combinarea MPI și OpenMP introduce overhead prin sincronizarea thread-urilor OpenMP și comunicarea între procesele MPI
- **Load balancing** - Distribuția neuniformă a calculelor poate apărea la ambele niveluri atât MPI cât și OpenMP
- **Sincronizare în etapa de Hysteresis** - Etapa DFS pentru edge tracking este recursivă, fiind dificil de paralelizat eficient

Flux date:

Master -> MPI Scatter -> OPENMP (CPU) -> MPI Gather -> Master

Profiling MPI-OPENMP

Identificarea Hotspot-urilor:

Elapsed Time: 4.673s

CPU Time: 13.110s

Total Thread Count: 36

Paused Time: 0s

Top Hotspots

This section lists the most active functions in your application. Optimizing these hotspot functions typically results in improving overall application performance.

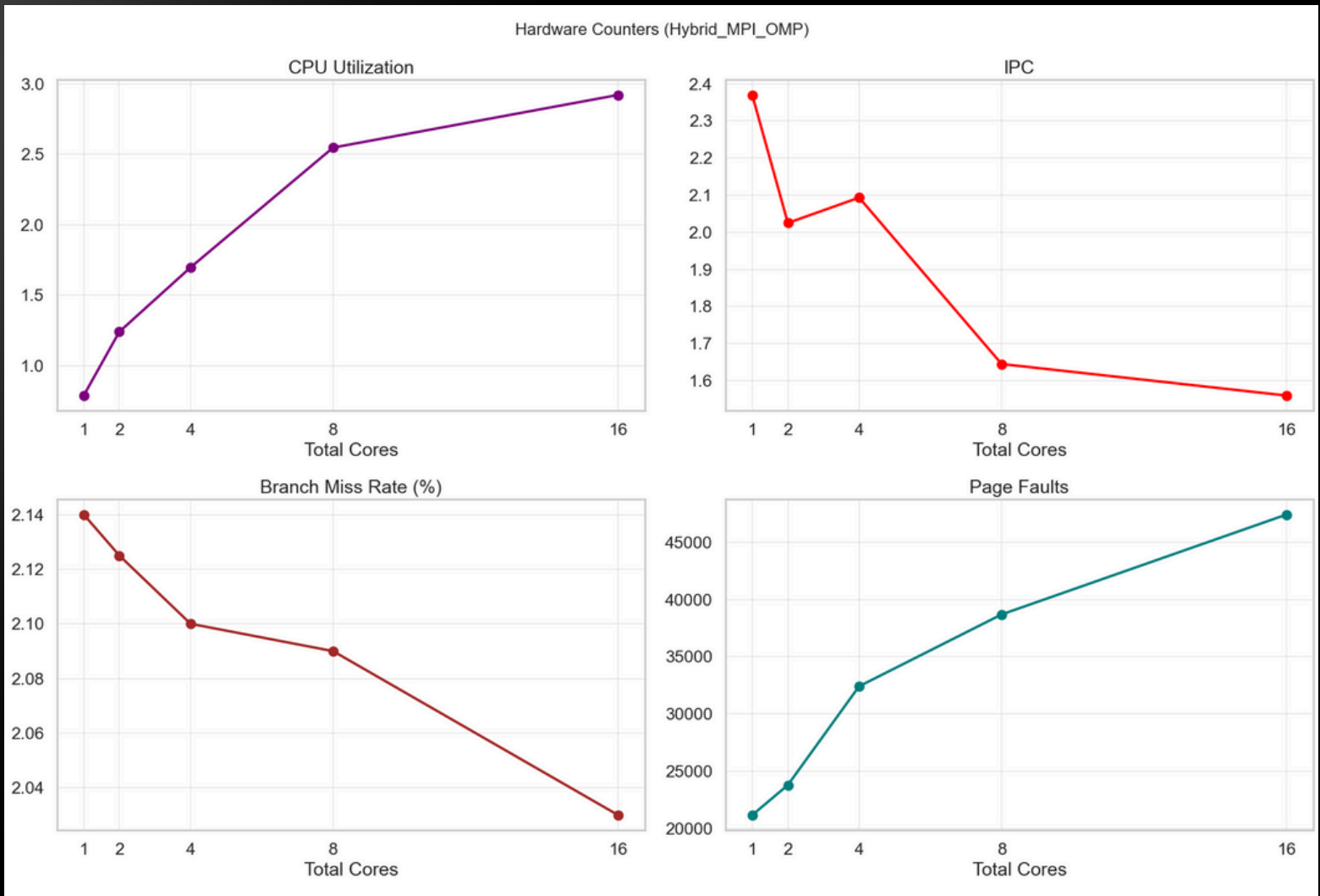
Function	Module	CPU Time	% of CPU Time
MPI_Bcast	libmpi.so.40	3.300s	25.2%
Image::gaussianBlur_omp_fn.3	Canny_hybrid_mpi_omp	1.100s	8.4%
atan2f	libm.so.6	1.090s	8.3%
sobelGradient_omp_fn.0	Canny_hybrid_mpi_omp	0.980s	7.5%
Image::gaussianBlur_omp_fn.2	Canny_hybrid_mpi_omp	0.930s	7.1%
[Others]	N/A*	5.710s	43.6%

*N/A is applied to non-summable metrics.

Performanțe hardware:

- **Tool folosit:** perf rulat pe xl
- **Eficiență (IPC):** Scade la 16 nuclee => saturație a benzii de memorie și overhead de sincronizare hibridă.
- **Predicție (Branch Miss):** Trend descendent (invers față de MPI) => execuția devine mai predictibilă pe măsură ce crește numărul de nuclee.
- **Memorie (Page Faults):** Creștere liniară => alocarea memoriei rămâne costisitoare din cauza naturii distribuite.
- **Utilizare CPU:** Scalare logaritmică bună până la 16 nuclee.

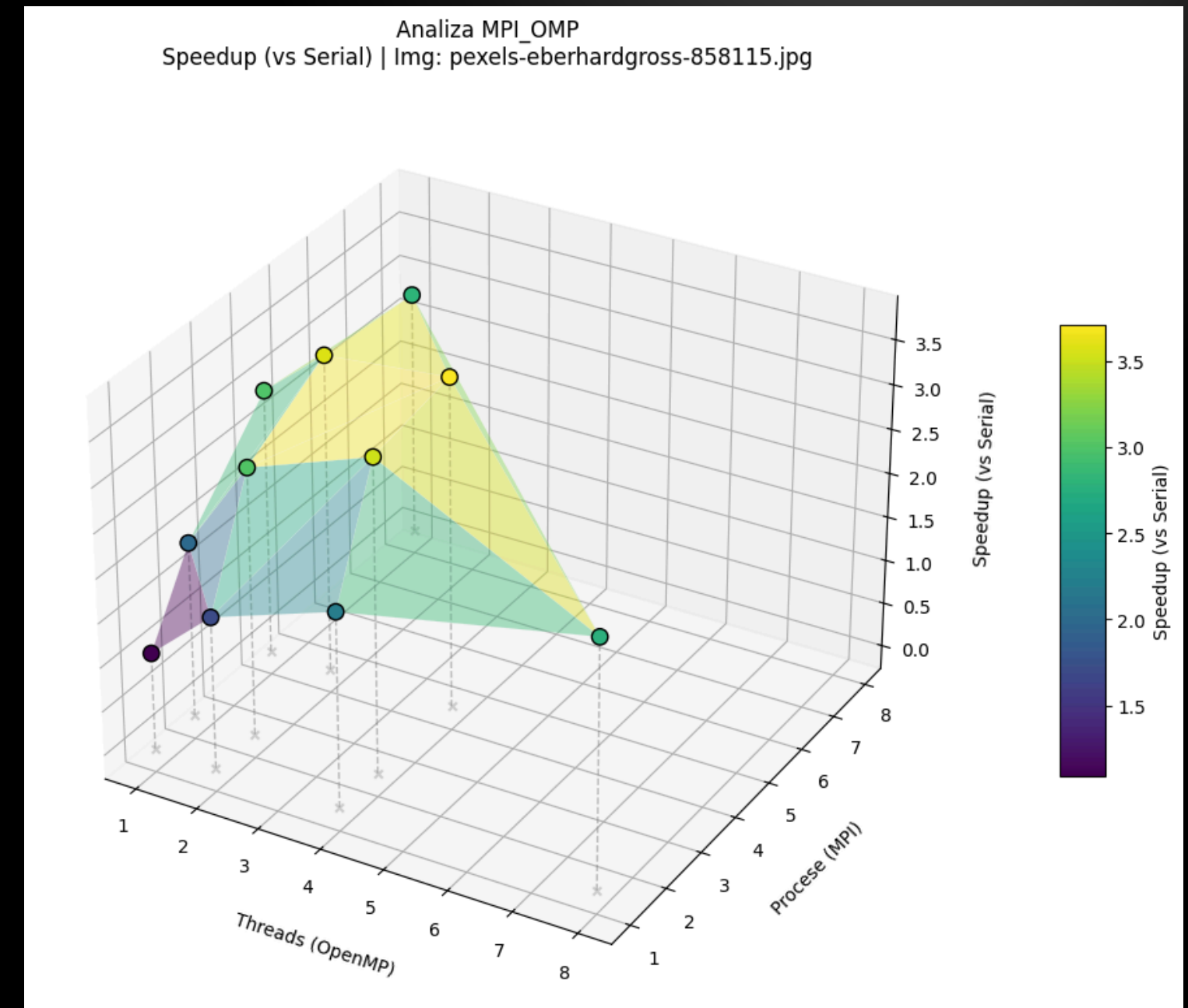
- **Tool folosit:** Intel VTune Profiler
- **Bottleneck:** MPI_Bcast (25.2%) => comunicarea rămâne costisitoare, dar impactul e redus față de MPI pur.
- **Calcul Hibrid:** Funcțiile _omp_fn (OpenMP) apar în top – paralelizare eficientă la nivel de thread.
- **Cost matematic:** Funcția folosită pentru calcularea direcției gradientului ocupa 8.3% => cost matematic constant.



Rezultate MPI-OPENMP

Rezultatul Optim:

- **Configurația "Sweet Spot":** Vârful graficului (zona galbenă) se atinge la combinațiile care totalizează 8 unități de execuție (ex: 2 Procese MPI × 4 Thread-uri OpenMP sau invers). Aceasta indică maparea ideală pe cele 8 nuclee fizice.
- **Speedup Maxim:** Se atinge o accelerare de aproximativ 3.5x față de varianta serială, depășind performanța MPI pur prin reducerea comunicării între procese.
- **Eficiență Hibridă:** Graficul demonstrează că performanța nu crește liniar doar adăugând resurse; optimul este atins atunci când produsul (Procese × Thread-uri) se pliază perfect pe numărul de nuclee fizice disponibile.
- **Saturație:** Zonele "joase" (mov/albastru) arată că subutilizarea sau suprasolicitarea (oversubscription) hardware-ului duce la scăderi drastice de eficiență.



Varianta MPI-Pthreads

Inovații & Diferențe:

- **Paralelism Hibrid:** MPI distribuie datele între noduri (Distributed), Pthreads le procesează local (Shared Memory).
- **Halo Rows:** Gestionare manuală a zonelor de suprapunere pentru convoluții corecte la marginile proceselor.
- **Memorie Eficientă:** Thread-urile împart același context (buffer-e), evitând duplicarea datelor per fir de execuție.
- **Sincronizare Ierarhică:** Bariere rapide locale (threads) în locul comunicării globale costisitoare.

Probleme întâmpinate:

- **Hysteresis Fragmentat:** Tracking-ul muchiilor (DFS) se întrerupe la granița dintre procesele MPI.
- **Complexitate Indici:** Gestionare dificilă a 3 niveluri de coordonate (Global, Local-MPI, Sub-Local-Thread).
- **Bottleneck I/O:** Etapele de Scatter și Gather rămân secvențiale și limitează scalabilitatea.
- **Load Balancing Rigid:** Distribuția statică a rândurilor ignoră densitatea reală a muchiilor din imagine.

Flux de date:

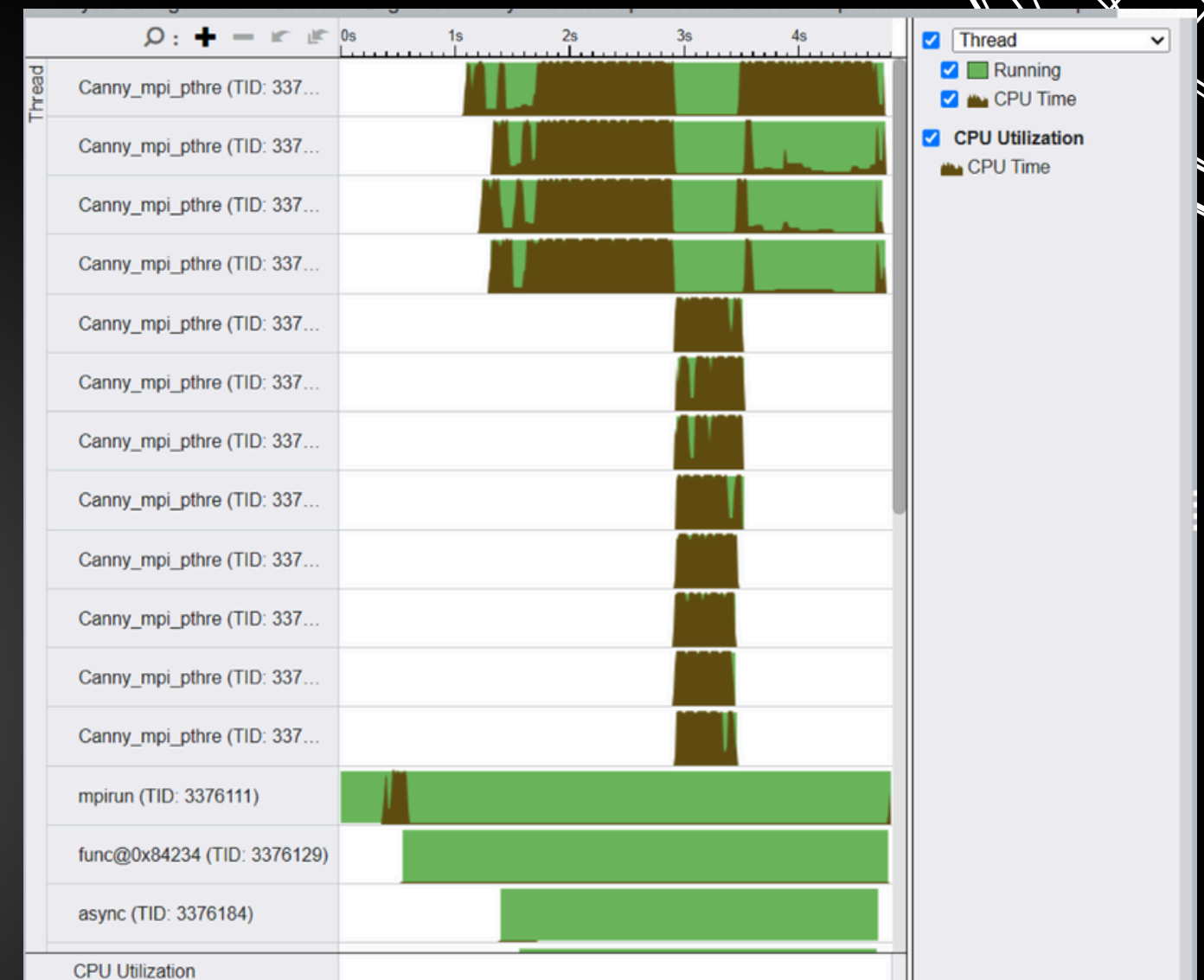
Master (Disk) -> MPI Scatter (+Halo) -> Pthreads (CPU) -> MPI
Gather (-Halo) -> Master

Thread

- ☒ Running
- ☒ CPU Time

CPU Utilization

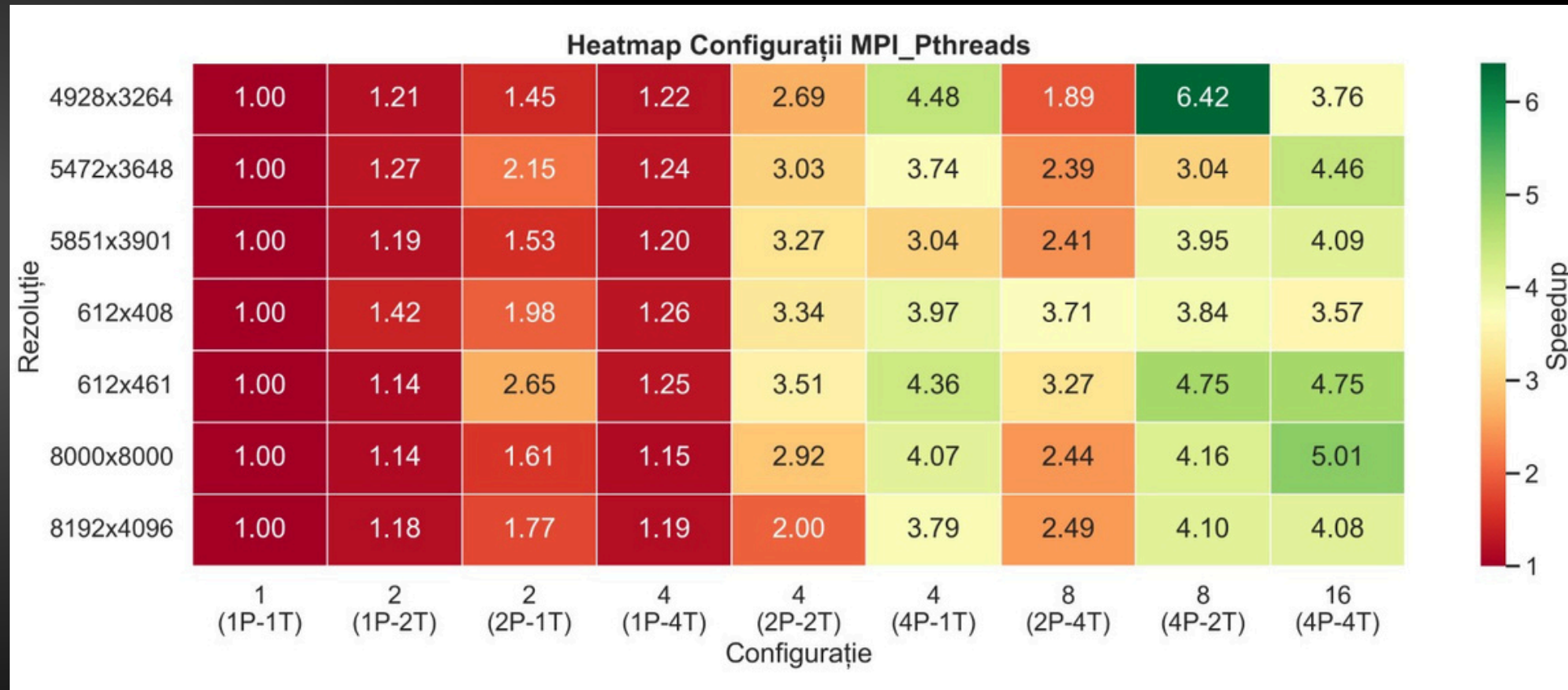
- ☒ CPU Time



A complex geometric pattern of overlapping white circles on a black background, resembling a sphere or a globe. The circles are arranged in a way that they create a sense of depth and curvature, with many lines intersecting to form a dense, web-like structure. The overall shape is roughly spherical, with the circles curving around a central point.

- **Costul Distribuirii (Overhead):** Funcția MPI_Bcast domină consumul cu 26.9%, reprezentând "taxa" de timp necesară transmiterii datelor către cele 4 procese.
- **Raport Muncă vs. Comunicare:** Timpul de calcul efectiv (workerThread - 24.9%) este comparabil cu cel de comunicare, ceea ce explică de ce scalarea nu este perfect liniară.
- **Configurație:** Rezultate obținute pentru 4 procese și 2 threaduri (partea de MPI este mai eficientă).

Rezultate MPI-Pthreads

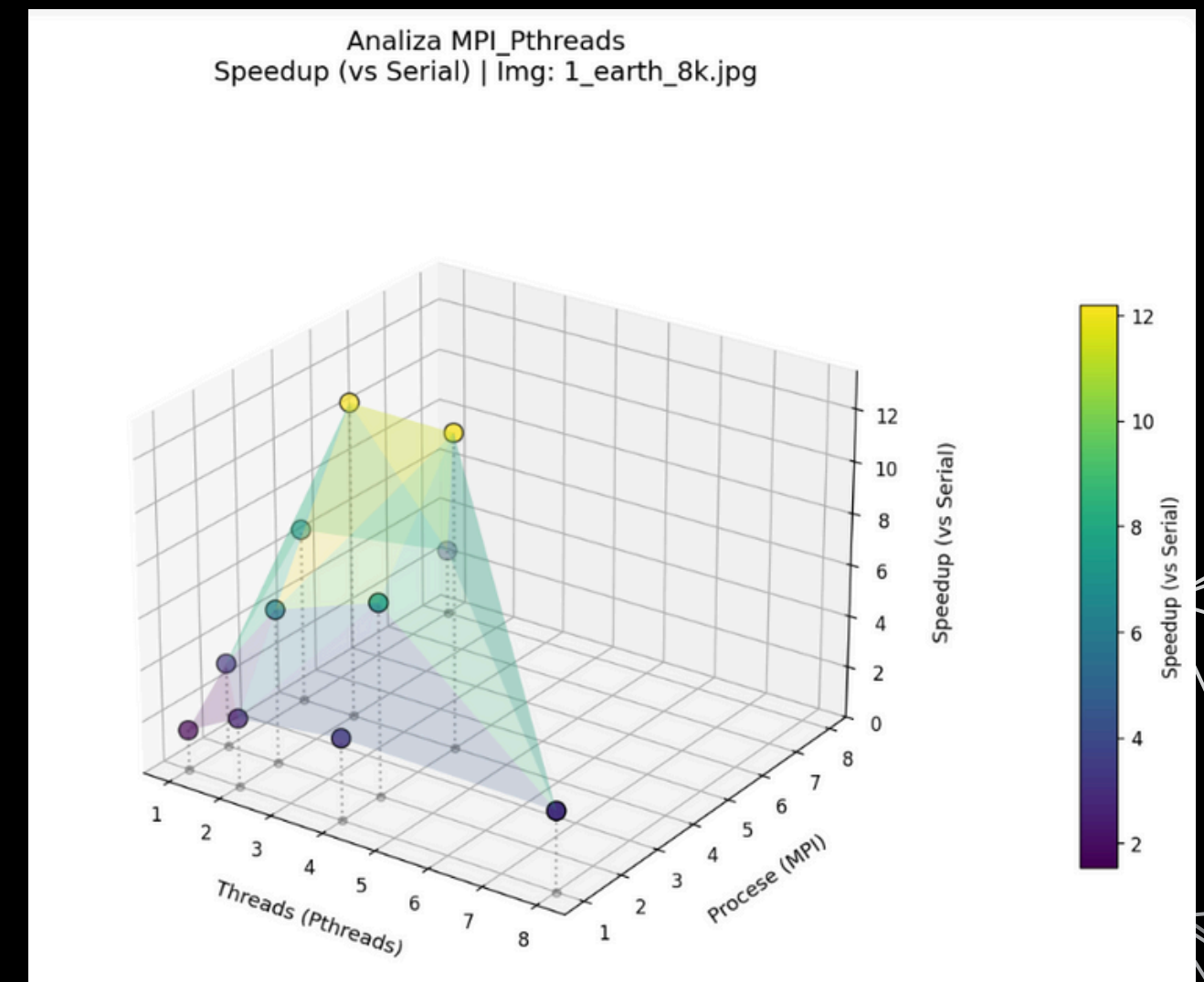


Superioritatea Proceselor (MPI):

- Comparația directă pe 4 unități de calcul este categorică: 4 procese MPI (4P-1T) obțin un speedup de ~4.4x, în timp ce 4 Thread-uri (1 proces și 4 threaduri) abia ating ~1.2x.
- Explicație: Procesele MPI au zone de memorie proprii, eliminând conflictele de scriere/citire în RAM care gâtuiau varianta Pthreads.

RECORD ABSOLUT: 6.42x

- Configurație: 4 Procese × 2 Thread-uri (Total 8 Unități)
- Rezoluție: 4928x3264 px
- Se obține pe o poză de dimensiuni medii
- Mixul de 4 Procese × 2 Thread-uri s-a dovedit cel mai eficient.
- Această combinație profită de memoria distribuită (MPI) pentru a împărți datele, dar folosește și agilitatea thread-urilor (Pthreads) pentru procesare locală rapidă.
- Eficiență de 80.25% pentru 4 procese și 2 threaduri.



Varianta CUDA + OpenMP

Inovații & Diferențe:

- **Paralelizare între imagini**, fiecare thread gestionează fluxul unei imagini distincte
- **Dynamic scheduling** pentru a compensa diferența de dimensiune dintre imagini (ex. imagini 612x408 și 8192x4096 în set)
- Fiecare thread își alocă **propriile resurse GPU**

Probleme întâmpinate:

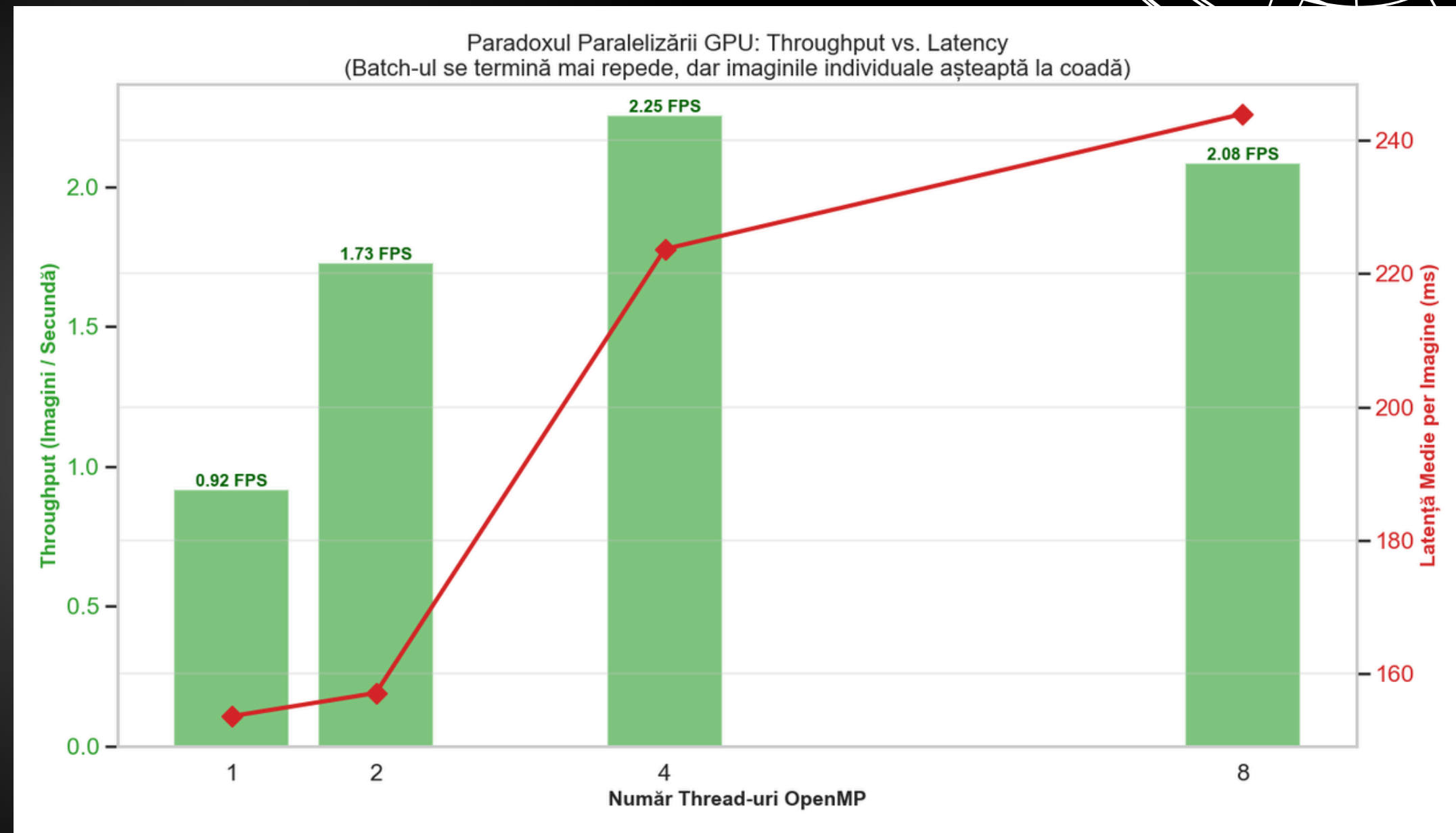
- **I/O bottleneck**, viteza de citire de pe disc limitează alimentarea GPU-ului cu date
- **Context switching**, overhead crescut când numărul de thread-uri depășește capacitatea GPU-ului
- **Oversubscription**, degradarea performanței la 8 thread-uri din cauza saturației resurselor

Flux Date (per thread CPU):

CPU (load & transfer) → GPU (procesare date) → CPU (final)

Profiling CUDA + OpenMP

- **Punctul optim** este atins la 4 threads, pentru throughput maxim
- Există un **tradeoff** între throughput și latența individuală per imagine
- **Degradarea rezultatelor** odată cu creșterea excesivă a threadurilor pe CPU



Complexitate Spațială: $O(H \times W \times T)$

Complexitate Temporală: $O(M \times I \times H \times W / (P \times T))$

H = image height, W = image width, M = num images,

T = num CPU threads, P = num GPU threads, I = iters hysteresis

Rezultate CUDA + OpenMP

- **Eficiența la nivel de pixel** este una constantă, vârful se menține la 16x16
- **Blocksize-urile** au un impact pozitiv asupra performanței, în timp ce **threadurile CPU** trag în jos eficiența

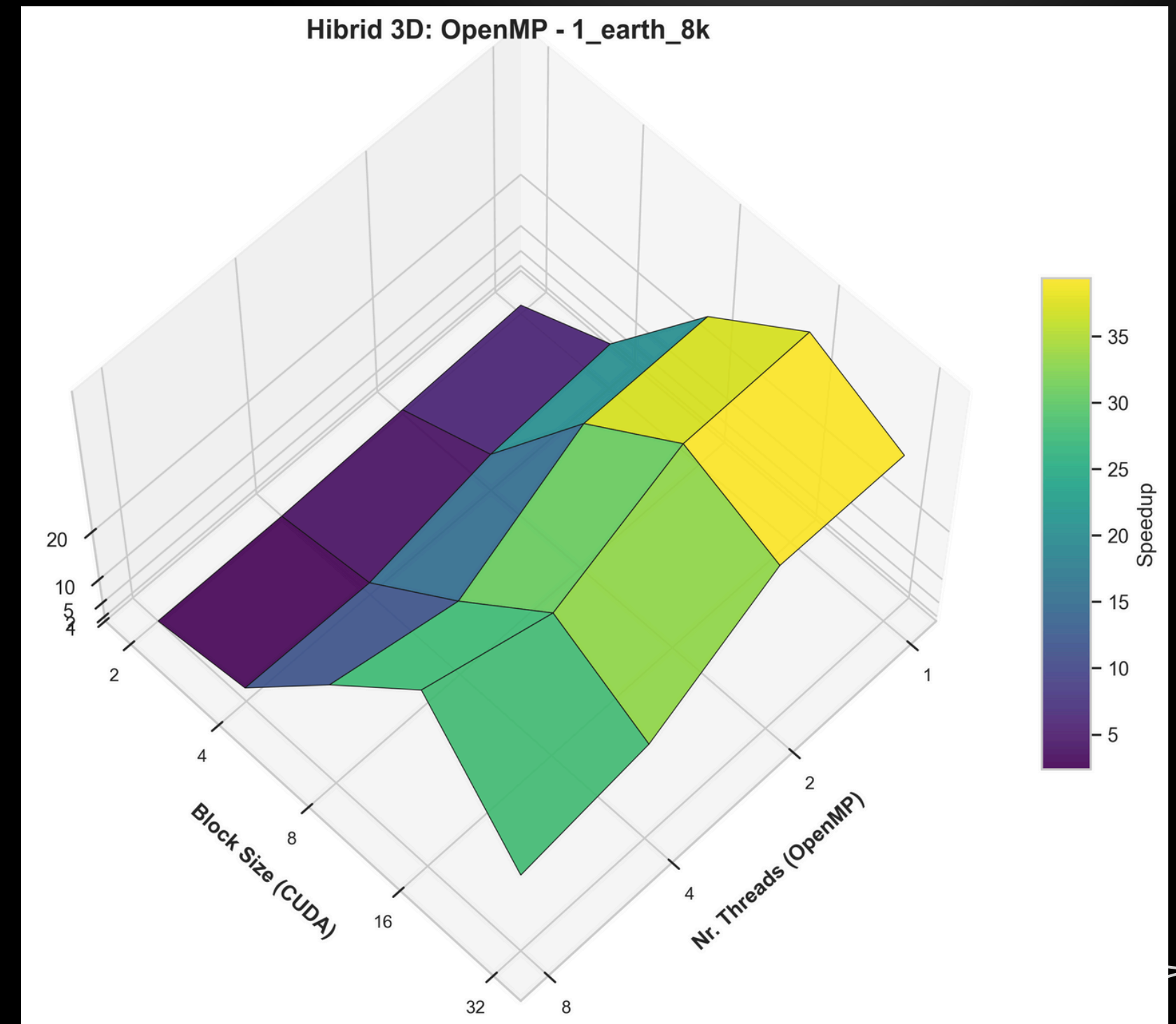
Configurație optimă (Latență): 16x16 (256 GPU threads per block), 1 CPU threads

Configurație optimă (Throughput): 16x16 GPU, 4 CPU threads

Cel mai bun timp obținut: 0.167s (0.53s pe întregul set de imagini), ambele pe configurația pentru latență

Speedup obținut: 45.3x

Eficiență obținută: 80.89% (56 cores)



Varianta CUDA + MPI

Inovații & Diferențe:

- **Descompunerea imaginii** între fâșii orizontale, distribuite între procese
- **Gestionarea zonelor de overlap** prin introducerea de rânduri suplimentare
- **Pipeline distribuit**, masterul citește și transmite datele către celelalte procese

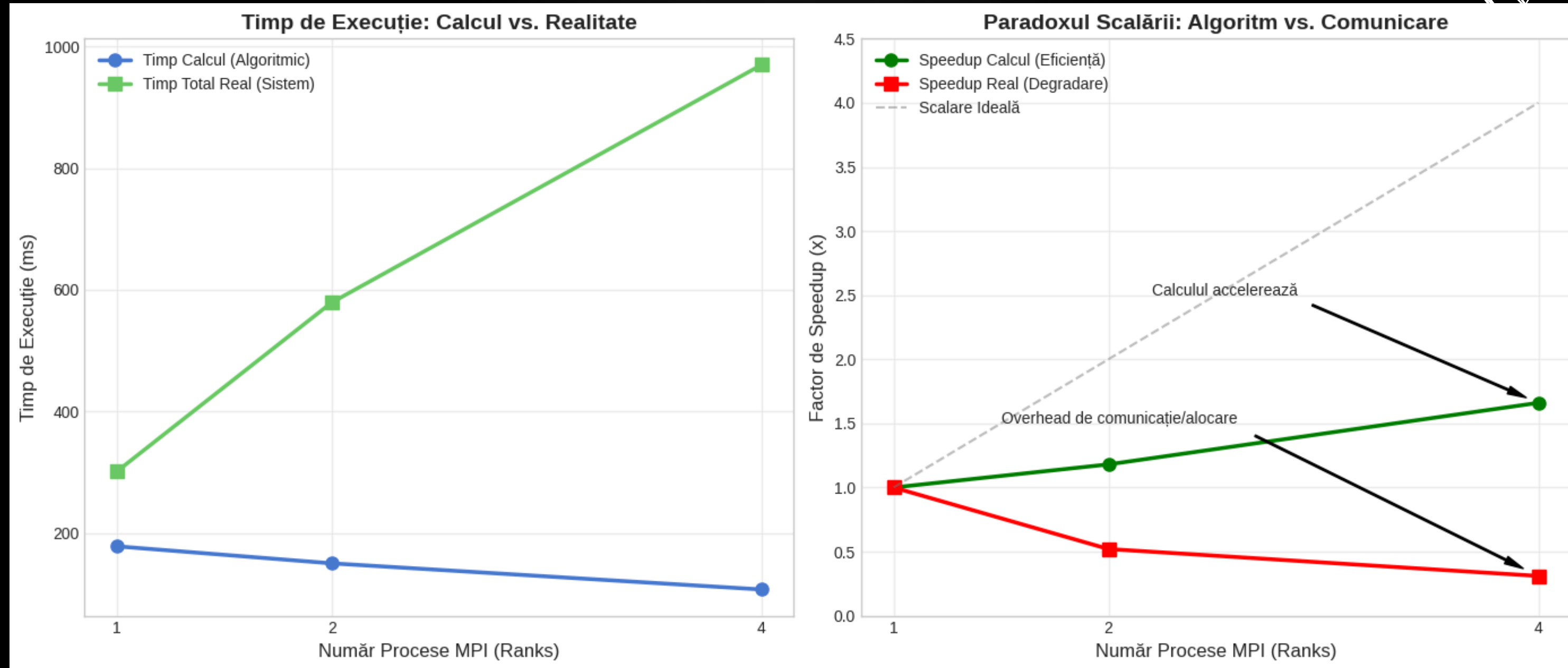
Probleme întâmpinate:

- **Calculul overlap-ului**, esențial pentru a evita artefactele sau erorile de memorie
- **Pierderea continuității**: în hysteresis, muchiile aflate la granițe își pierd urma
- **Overhead de comunicație**, costul transferului depășește beneficiul paralelizării la imagini mici

Flux Date:

CPU (load & transfer între procese) → GPU (procesare fâșie per proces) → CPU (asamblare imagine finală)

Profiling CUDA + MPI



- **Timpul de calcul** scade constant (speedup 1.66x la 4 procese)

- **Overhead-ul de comunicație** domină timpul total de execuție (speedup real subunitar, 0.31x)

Rezultate CUDA + MPI

- **Performanța** scade gradual, odată cu creșterea numărului de procese
- Graficul confirmă faptul că **performanța maximă se atinge la blocksize-ul 16x16**

Configurație optimă (timp total): 16x16 (256 GPU threads per block), 1 proces MPI

Configurație optimă (timp calcul): 16x16 GPU, 4 procese

Cel mai bun timp obținut: 0.167s

Speedup obținut: 45.3x

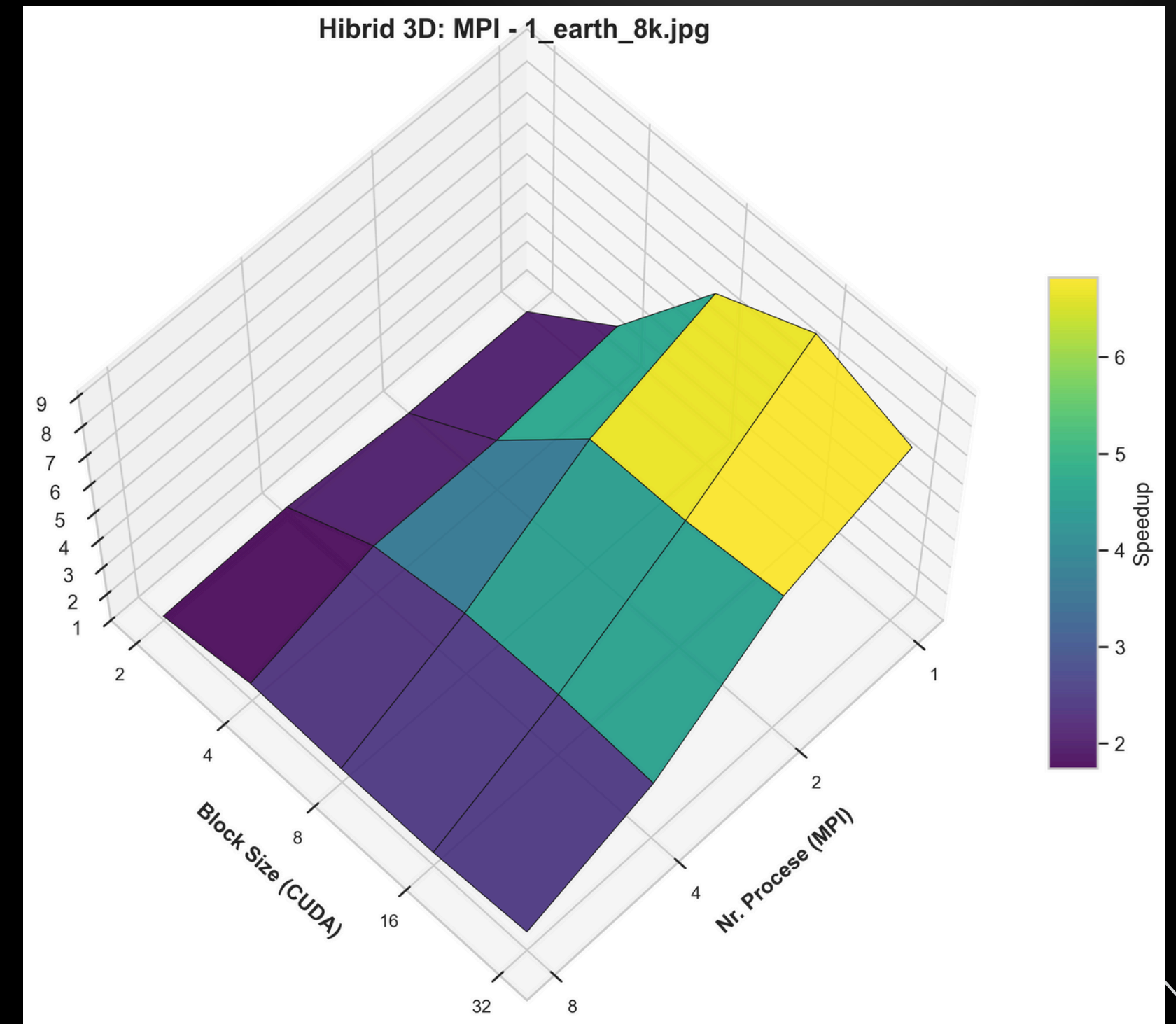
Eficiență obținută: 80.89% (56 cores)

Complexitate Spațială: $O(H \times W \times S)$

Complexitate Temporală: $O(H \times W / (P \times S))$

H = image height, W = image width,

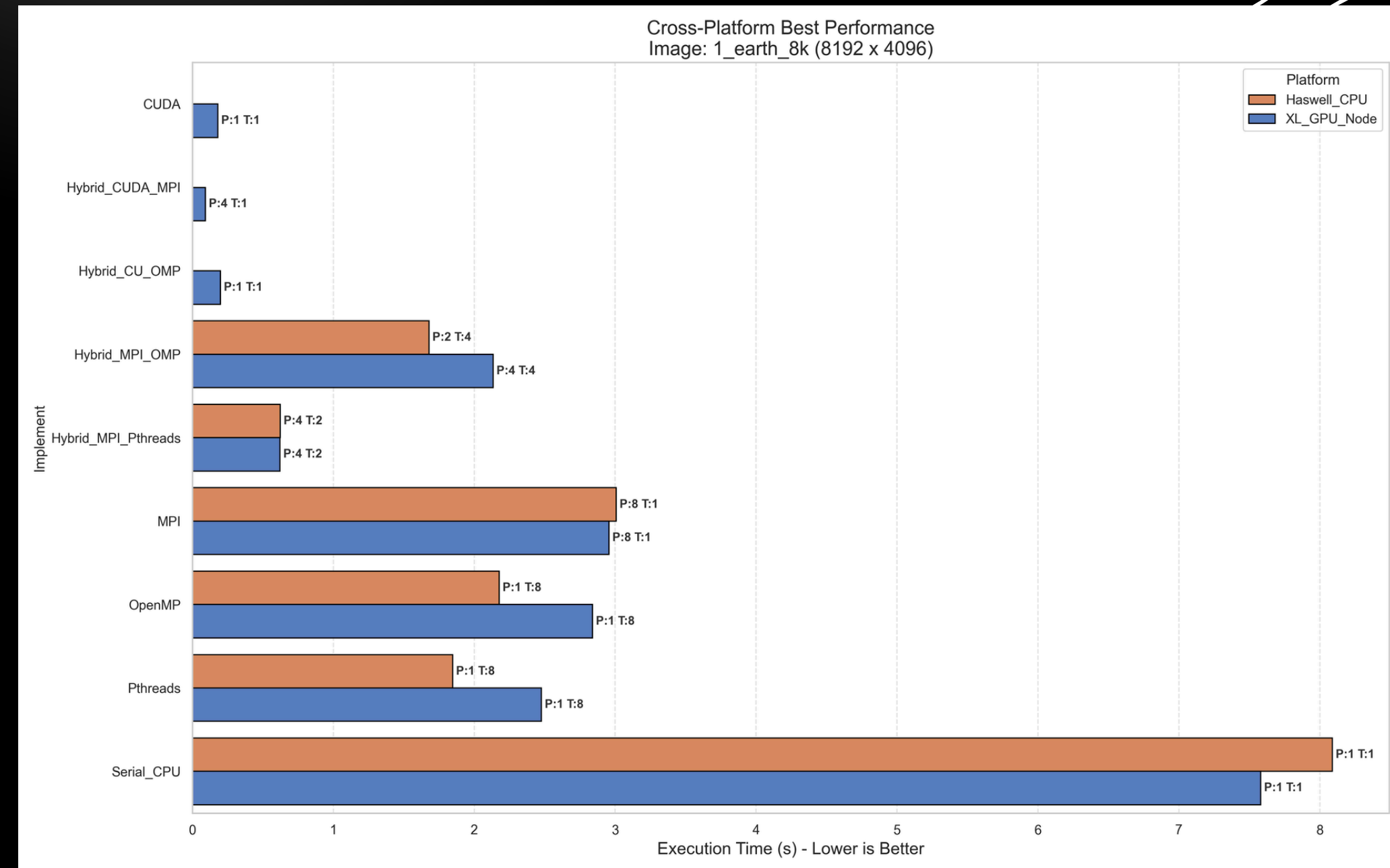
P = num procs CUDA, S = num procs MPI



Analiza Comparativa

Clasificarea pe niveluri:

- **Tier 1 (GPUs):** variantele CUDA au cea mai bună performanță, cu o eficiență de aproximativ 80%
- **Tier 2 (CPU optimizat):** implementările hibride și cele ce folosesc shared memory oferă cei mai buni timpi pe CPU
- **Tier 3 (Baseline & Overhead):** implementarea MPI suferă din cauza overhead-ului de comunicare

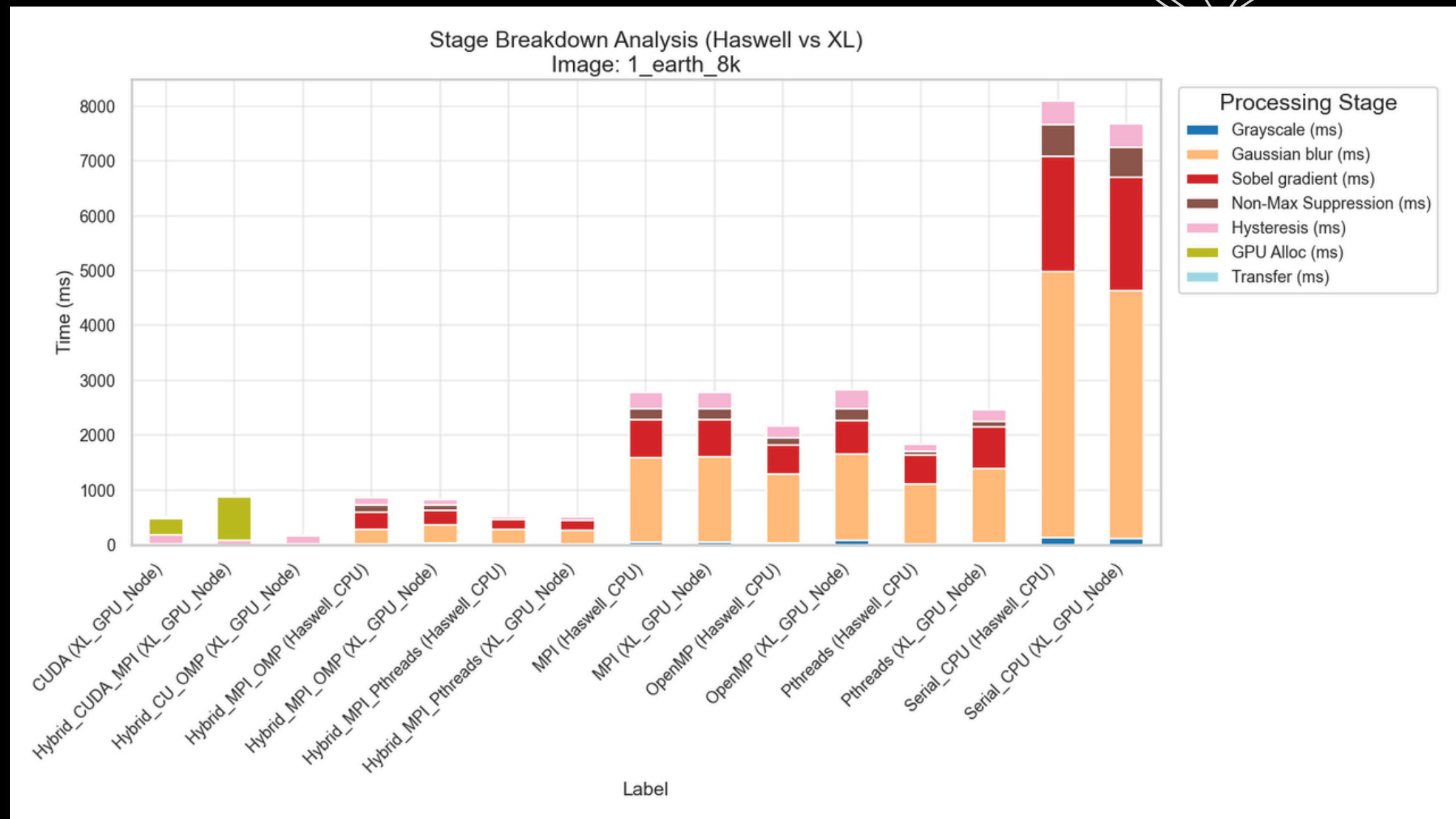


*timpul mapat în acest grafic pentru Hybrid CUDA+MPI este timpul de calcul, timpul total fiind cel mai slab dintre implementările GPU

Breakdown Etape

CPU vs GPU:

- **CPU:** timpul este dominat de calcule matematice grele (ex. Gaussian Blur și Sobel)
- **GPU:** aceste etape dispar total, timpul este dictat de Hysteresis (fiind iterativ) și de transferul de date



Concluzie

Nu există o soluție universală. Alegerea arhitecturii depinde de constrângerile hardware și de volumul de date:

- pentru performanță → CUDA
- pentru procesare batch → CUDA + OpenMP
- pentru imagini gigantice → CUDA + MPI
- clustere CPU (fără GPU) → MPI + Pthreads
- simplitate în dezvoltare → OpenMP



**MULTUMIM PENTRU
ATENTIE!**